



PUBLIC

MAIF

Document Version: 2402 – 2024-05-08

Mobile Application Integration Framework Developer Guide

Content

1	Mobile Integration Framework for SAP.	4
1.1	General Configurable Behaviors.	5
1.2	SAP Framework.	9
1.3	Mobile Exchange Persistent Layer.	11
1.4	Delta Detection Routine.	12
1.5	Change Detection Configuration Set.	12
1.6	Data Object Handler Class Repository.	12
1.7	Mobile Integration Configuration Set.	13
1.8	Application Authentication Services.	13
1.9	BAPI Wrapper.	13
1.10	System Monitor.	14
1.11	Configuration Module.	14
1.12	Working with Push Scenarios.	14
1.13	Outbound Trigger Handling.	15
2	Overall Synchronization Processing.	16
2.1	Fetch Synchronization.	16
2.2	Complex Table Synchronization.	18
2.3	Data Table Synchronization.	19
2.4	Server Push Synchronization.	21
2.5	Transaction Synchronization.	22
3	Using the Mobile Data Objects (MDOs).	24
3.1	BAPI Wrappers.	24
3.2	Data Tables Overview.	27
3.3	Complex Tables Overview.	30
3.4	Standard Data Objects Overview.	40
4	Change Detection.	43
4.1	xChange Framework Overview.	43
4.2	EFI Development.	46
4.3	Exchange Table Development.	48
4.4	Lock Object Development.	49
4.5	Exchange Handler Development.	49
4.6	Exchange Object Configuration.	54
4.7	EFI Configuration.	57
5	GIS Integration.	59

6	Setting Up Data Staging.	62
6.1	General Steps to Activate Data Staging.	62
6.2	Data Staging Activation: Detailed Example.	65
6.3	Troubleshooting, FAQs, and Resources.	77
6.4	Data Staging Flows and Predefined Tables.	80
7	Inbound Transaction Management.	83
7.1	Inbound Transaction Queue Configuration.	83
7.2	BAPI Wrapper Enablement.	85
7.3	Inbound Transaction Processing.	85
7.4	Inbound Transaction Administration.	90
7.5	Inbound Transaction Management Security.	91

1 Mobile Integration Framework for SAP

The Mobile Add-On is an ABAP add-on component to SAP. This component provides the business logic related to the synchronization of data between SAP and the SAP mobile application. It provides integration for data synchronization between SAP and the Agency-based mobile application. The Mobile Add-On contains default synchronization logic for the product with which it is bundled.

The Mobile Add-On is comprised of two main layers, with multiple elements contained within each layer: the change detection layer and the integration layer. The Mobile Add-On also includes both a Configuration Panel and an Administration and Monitoring portal.

Change Detection Layer (xChange Data Model)

The change detection layer of the Mobile Add-On provides support for the xChange data model of synchronization. It contains the xChange tables and triggers used to detect and track changes made to data in SAP and stores this information, also known as xChange data. Most processing occurs independently of the mobile clients.

The change detection layer only tracks data that is of importance to the mobile application through the use of assigned fields, triggers, and linkages. Information recorded in the change detection layer is used during the downstream synchronization of the mobile application.

The change detection layer includes the following elements:

- xChange Data Objects: Accessed through the ConfigPanel, the exchange data objects encapsulate the xChange tables, class handlers, and other components involved in the change detection process.
- EFI Triggers: Created in the SAP enhancement framework, triggers detect changes to production data of importance to the mobile application.
- Exchange Tables: Updated by the xChange data objects to track changes to data of importance to the mobile application. This xChange data is used during synchronization of the mobile application client.

Change detection processing occurs according to the following general steps:

1. A change is made to a record in an SAP table.
2. The EFI trigger for the SAP table notifies the assigned exchange object of the change and the type of change:
 - **Create**: Change notification includes the entire record created.
 - **Update**: Change notification includes both old and new records, and the fields that were modified in the SAP table.
 - **Delete**: Change notification only includes the unique identifier.
3. The xChange object evaluates the changes against the active, or selected, change detection fields and against the change detection filters. If the changes are not against active selection fields or filters, no further processing occurs.
4. If the change is against active selected change detection fields or filters, the xChange table is updated by the xChange object.

5. If linkage is set for additional xChange objects, those objects are notified and the change detection processing flow starts for those objects, beginning at step 3.
6. If the EFI trigger is assigned to multiple exchange objects, this entire process is repeated for each notified xChange object.

Centralized Configuration (ConfigPanel)

The Configuration Panel is the main configuration component of the SAP application. Data is mapped to the SAP application and pulled from SAP modules based on the user's credentials. Therefore, administrators can allow remote workers to navigate and see only data relevant to their job location or function.

Centralized Administration and Monitoring Portal (Admin Portal)

The Administration and Monitoring portal is provided to allow administrators the ability to manage the production system. It includes monitoring and logging functionality, as well as problem-detection tools utilizing SAP's standard application log database. It allows administrators to trace who is logging into the mobile applications, what work they are doing, and the data being transferred between the mobile devices and SAP.

Integration Layer

The integration layer provides an interface and business logic for both upstream and downstream data synchronization.

In upstream synchronization, information captured on the Client is transmitted to the Server. The integration layer includes business logic and processing to update the appropriate data in SAP. In downstream synchronization, SAP uses information in the exchange tables from the change detection layer to determine what data a given mobile client needs by comparing the information in the exchange table records with the information existing on the mobile client. The nature of the client information will vary depending on the Agency definition type being synchronized.

1.1 General Configurable Behaviors

Many basic mobile application behaviors are configurable through the Mobile Add-On, in addition to the ability to make changes through SAP or the Agency Editor. The basic configurable behaviors are described below.

Mobile Data Objects (MDOs)

- **Field Retrieval** - Field retrieval is configured for mobile data objects by customizing field usage settings to only retrieve required data for the mobile application through the GET method of the class handler. Once the field retrieval is configured, further configuration of data filtering and change detection of the fields can be performed.
- **Data Filtering** - Data filters within mobile data objects control what data can be viewed by a specific mobile application, providing the functionality to restrict data access for those mobile applications. Data filters can be user-dependent or applied to an entire mobile application. It is important to define proper filter rules in order to distribute only necessary records to the client application. Each user is assigned a role-based profile with authorization restrictions for viewable data and activities they can perform on the Client. For example, a user working in one location should not be able to view data for a different location, unless it pertains to their job. In this example, filters are used to restrict data to a user's specific location.
- **Data Object Handler Class Repository** - The data object class handler repository links BAPI wrappers with mobile data objects and encapsulates all business logic related to mobile applications. Internally standard BAPIs or custom logic can be implemented to provide optimal mobile support. The object-oriented design provides benefits such as reliability, reusability, extensibility and maintainability.

All transaction updates are performed through standard BAPIs in order to ensure data integrity. The repository also supports streamlining multiple SAP transactions or multiple BAPIs for improved business processes. The data objects within the repository are application-dependent and delivered in the relevant package based on the SAP Business Suite Application hierarchy, such as */SMERP/MM* or */SMERP/PM* for ERP, or */SMCRM/MM* or */SMCRM/PM* for CRM.

The mobile data objects contained within the repository support result-set field selection, data filtering and security checks as defined by the integration rules. The mobile data objects also support monitoring and logging applications. Mobile data objects are configurable through the ConfigPanel. Different rules can be defined for every class handler for each mobile application.
- **Mobile Integration Configuration Set** - The Mobile Integration Configuration Set within the business logic layer contains the user interface components presented in the ConfigPanel. The tools within this configuration set allow for the creation, configuration, and administration of the components within the business logic layer.
- **BAPI Wrapper** - The BAPI wrapper layer provides a consistent way to expose business logic and data from the SAP system to external mobile applications. The BAPI wrappers are decoupled from the business logic in SAP, making them easy to reconfigure or customize.

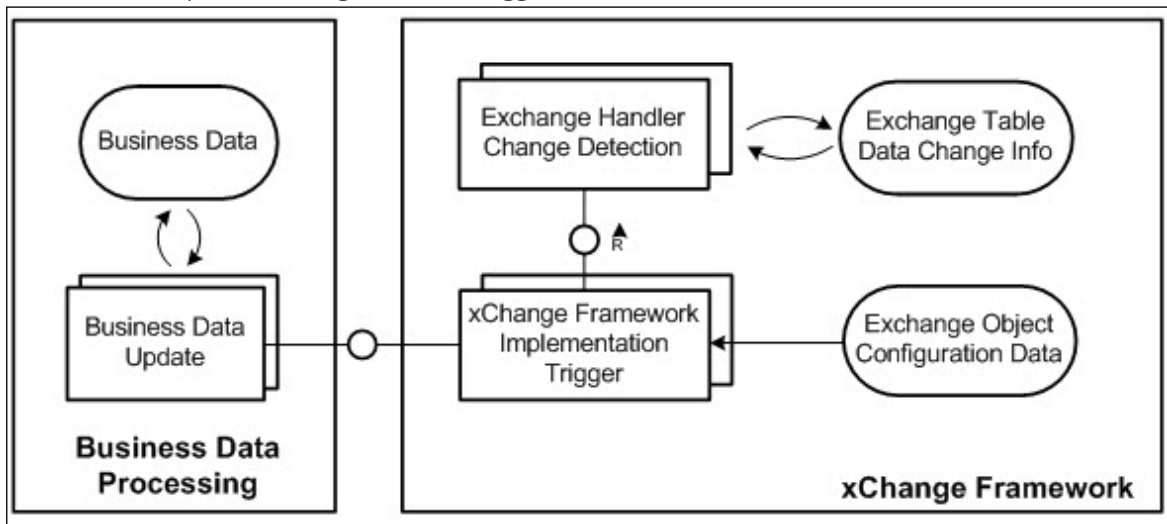
The standard BAPI wrapper library consists of a collection of custom developed BAPIs based on the SAP integration framework. BAPI wrappers improve interface consistency with standard naming conventions and development standards. The BAPI wrapper library is application-dependent and delivered in the relevant package based on the SAP Business Suite Application hierarchy, such as */SMERP/MM* or */SMERP/PM* for ERP, or */SMCRM/MM* or */SMCRM/PM* for CRM. BAPI wrappers set parameters for mobile data objects to post and pull data in the SAP system. When the Server calls to SAP, these calls are encapsulated in a BAPI wrapper to ensure that SAP responds in a consistent manner and performs the actions that SAP is asking for.

Use the ConfigPanel to configure BAPI wrappers by assigning them to mobile data objects and handler methods (*GET*, *CREATE*, *UPDATE*, or *DELETE*), function groups, and packages within SAP.
- **System Monitor** - Administration and monitoring activities that take place behind the SAP Java connector for the SAP are performed through the System Administration and Monitoring Panel. The Administration Panel is a problem-detection tool utilizing SAP's standard application log database. It allows administrators to trace who is logging into the mobile applications, what work they're doing, and the data being transferred between the mobile devices and the SAP system. The Administration Panel speeds problem resolution using real-time data and traceability.

- **Configuration Module** - The Configuration Module is the SAP configuration Web interface panel. The Configuration Panel simplifies modifications to existing mobile applications. It adds a layer of business logic for filtering at a system or user level. Data is mapped to SAP and pulled from SAP modules based on the user's credentials in the SAP system. Therefore, administrators can allow remote workers to only navigate and see data relevant to their job location or function.

xChange Data

- **Change Detection Fields** - Change detection fields are fields within the tables of the SAP system that are of concern to the mobile application. The exchange object contains a list of these fields that meet this criteria. When a record is changed, that change must include a modification to one or more of these fields. If it does not, the change is ignored by the exchange object. If it does include one or more of these fields, the change is processed by the exchange object. The lists of fields the exchange object considers important can be modified, including adding fields to the list or removing and deactivating previously active fields from the list. Typically, fields are added to the list of change detection fields when those fields are added to a mobile data object's result set field selection.
- **Change Detection Filters** - Change detection filters provide additional abilities to restrict change detection based on data content in exchange object fields. Using change detection filters, you can define filter conditions for which the underlying business object must qualify before the change detection process is triggered.
- **Mobile xChange Persistent Layer** - The mobile xChange persistent layer allows the transfer of data from SAP to the mobile client as well as the transfer of data from the mobile client to SAP. This xChange process provides a consistent way to manage and capture mobile client-related master and transaction data changes within SAP. The mobile xChange persistent layer uses the SAP NetWeaver enhancement framework to implement change detection triggers.



- **Delta Detection Routine** - The delta detection routines are the triggers within the enhancement framework that detect modifications to data of concern to the mobile application. When changes are made to production data in the back end system, the delta detection routines will capture the change event. If a given change is one that will affect the data in the mobile application, then the routine will update the appropriate xChange table in the mobile exchange persistent layer. Intelligence is incorporated in the delta detection routines to determine if a change is one of concern to the mobile application. If it is not, then the change is ignored by the routine.

- **Change Detection Configuration Set** - The change detection configuration set controls how the change detection should be carried out by the mobile xChange persistent layer. Customer-defined configurations are protected through a reserved customer namespace, such as "Y*" or "Z*". The change detection configuration set is mobile application-independent. However, it is a set of mobile application-specific change detection rules. The rules are configurable through the ConfigPanel.

Middleware

- **Client Global Settings** - The Client Global settings replace some global definitions previously found in the Agentry project with globals that are configurable in the Configuration Panel. Globals configured through the Configuration Panel will start with a group name of either "DT" (data table) or "CT" (complex table) and are brought down to the Client. All globals accessed through the Configuration Panel are of string data type.
Globals in the complex table group are used for CATS time management.
If the group name does not start with either DT or CT, they are not stored on the Client and are only used for synchronization logic. These globals are still stored in Agentry. If a global is referenced by an attribute of another definition in Agentry, it will be stored in Agentry as well, and is not accessed through the Configuration Panel.
- **Parameters** - Use the parameter settings to configure parameters necessary to synchronize complex tables and data tables in the mobile application.
- **Push Notifications** - A push scenario pushes emergency work orders to the corresponding recipients. Following is the basic flow of a push instance from generation in SAP to reception on the client.
 1. The push exchange process initiates the push trigger based on the push conditions. Conditions are defined as filter rules in the push exchange object. For instance, **work order priority = 1** is considered an emergency work order in the base product release.
 2. The work order that satisfies the push conditions inserts a record into the push register table /SYCLO/PSH01 with an object key as the work order number and a push status of **NEW**.
 3. The event /SYCLO/BACKGROUND_JOB_EVENT is raised after the work order is saved, which triggers the background job for the push processor agent.
 4. The push processor job /SYCLO/CORE_PUSH_PROC_PROG is triggered, either by using the event or the time frequency. This trigger is based on specific customer processes.
 5. The push processor determines the recipients for the push work order and builds the data for each recipient as a separate instance. The instance is stored in the outbound message queue /SYCLO/PSH02 with queue **ID = PUSH**, using the staging database.
 6. The push instance displays one of the following statuses, viewable in the push monitor in the Admin Panel: **NEW**, **PROCESS**, **CANCEL**, **COMPLETED**, or **SRV_COMP**.
 7. The Agentry application within the SAP mobile server calls the push BAPI /SYCLO/PM_DOPUSHWORKORDER_GET for every predefined time interval and checks the push queue for new items.
 8. The Agentry application within the SAP mobile server sends the push data to the respective clients depending on the user credentials that match the push instance.
 9. Once the client receives the push message, it sends the client confirmation back to the server and the server calls the BAPI /SYCLO/CORE_PUSH_STAT_UPD to update the confirmation with status **CLNT_CONF** back to SAP.
- **GIS Service** - A geographic information system (GIS) integrates hardware, software, and data for capturing, managing, analyzing, and displaying all forms of geographically-referenced information. The SAP Work

Manager and SAP CRM Service Manager applications have custom map controls with GIS functionality implemented using SMP 3.0 Agentry SDK and Open UI extensions. Their functionality is configured through the Agentry application definitions. Both mobile applications are delivered with pre-defined mobile parameters that are ESRI-specific, however, you have the option to point to any GIS vendor you choose.

- **Data Staging** - During the initial sync time of any Agentry application, the bulk of the time is spent downloading the complex table required for the application. Depending on the data volume of the table, this sync time can be significantly high. By enabling data staging, the synchronization time for a very large data set can be reduced.

Data staging is only supported for complex tables in standard product. By default, data staging is turned off. The use of data staging is only recommended for initial sync, dealing with static data to reduce time during initial synch and any errors due to data staging being out of sync. It is recommended that you take advantage of the xChange framework for subsequent master and transactional data delta syncs.

- **Inbound Transaction Management** - Inbound transaction management provides the functionality of asynchronous transaction processing. This functionality allows mobile client transactions to be uploaded and checked into inbound transaction queues in the SAP system. Transactions parked in the inbound transaction queues are processed asynchronously at a later time.

Inbound transaction management can be used for regular transaction processing or transaction error handling. Inbound transaction management can be enabled based on object type. A hybrid mode of mixing asynchronous transaction processing with synchronous transaction processing based on object types is also supported.

Synchronization

Configuration changes can be made to mobile data objects, push scenarios, and EFI triggers, and BAPI wrapper assignments can be changed, all to affect the overall synchronization behavior of the SAP application at run time. Some of these changes are made only in the elements of the Mobile Add-On, while others require additional changes in the Agentry application project for SAP.

1.2 SAP Framework

The framework consists of core components. These components are logically grouped in three main layers:

- Change detection layer
- Business logic layer
- Integration layer

Within the framework, there also exists the configuration module and system monitor. The configuration module provides the interface to the integration framework to allow for the configuration and administration of the various components within it. The system monitor provides an administrative interface to monitor processing related to users, pushes, and other synchronization tasks within the framework.

Change Detection Layer

The change detection layer of the Mobile Add-On contains the xChange tables and triggers used to detect and track changes made to the data in the SAP system, and stores xChange information used during synchronization between the mobile application and SAP. Included in the change detection layer are the following:

- *Mobile Exchange Persistent Layer* - Contains the exchange tables used during synchronization to compare data on the Client to data in the SAP system, and to support the synchronization of only the differences.
- *Delta Detection Routines* - Contains the triggers, created in the SAP enhancement framework, to detect changes to production data of importance to the mobile application.
- *Change Detection Configuration Set* - Configuration tools, including the interface presented in the Configuration portal, to allow for the creation, configuration, and administration of the components of the change detection layer.
- *Exchange Object* - Presented in the ConfigPanel, the exchange object encapsulates the SAP tables, xChange table, class handler, and other components involved in the change detection process. Exchange objects are utilized by the fetch process to determine what data has changed since the last transmit of the mobile device.

Business Logic Layer

The business logic layer contains the logical components to work with the xChange data objects in the change detection layer for downstream synchronization, as well as the logical components to update data from transactions in the mobile application to the SAP system. Included in the business logic layer are the following:

- *Application Data Filter Services* - Specifies the filtering of data to be retrieved from the SAP system for transmission down to the mobile application. Not all fields from a given table will necessarily be sent to the mobile application. The application data filter services allow for the specification of the specific fields from a table to be sent to the mobile application.
- *Data Object Class Handler Repository* - Contains the logic used by the mobile data objects to synchronize data. A class handler is created to retrieve data from or update data to the SAP system, making calls to the standard SAP BAPIs.
- *Application Authorization Services* - Contains the security settings specific to the mobile user group. Settings for authorization services can be applicable to users of all mobile applications synchronizing data through the Mobile Add-On, for a specific application, or for a specific class handler within the business logic layer.
- *Mobile Integration Configuration Set* - Configuration tools, including the interface presented in the Configuration Panel, to allow for the creation, configuration, and administration of the components of the business logic layer.
- *Mobile Data Object* - Presented in the ConfigPanel, the mobile data object encapsulates the class handler, xChange object (in the change detection layer), the application data filtering services, and other components involved in the synchronization of data. The mobile data object references the xChange object in the change detection layer to support its synchronization. It in turn is referenced by the BAPI wrappers within the integration layer.

Integration Layer

The integration layer of the Mobile Add-On contains the BAPI wrappers that present the integration point to the mobile application servers. The Java steplets, complex table, and data table classes within the synchronization definitions of the mobile application call into these BAPI wrappers, through the SAP Java Connector (SAP JCo). The BAPI wrappers, in turn, call into the mobile data objects, which include the business logic for synchronization.

The architecture and design of the BAPI wrappers is intended to exclude the business logic related to the synchronization of the production data. This structure minimizes the necessity of modifying Java code within the mobile application as the result of changes to the synchronization processing configured in the integration of the Mobile Add-On.

Configuration Module

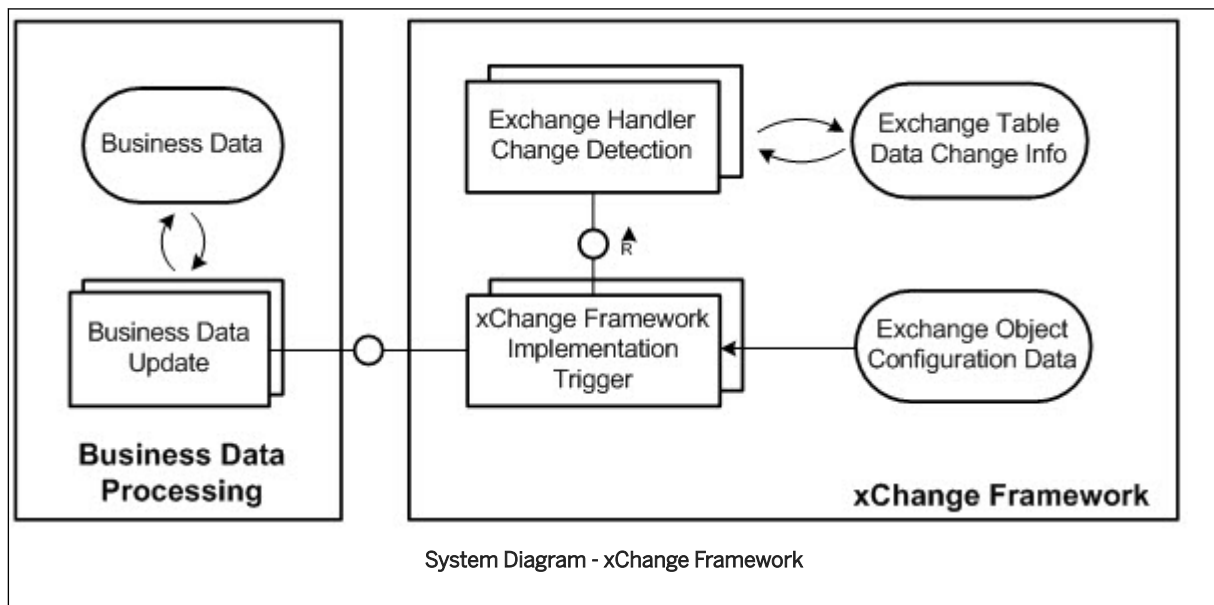
The configuration module is the Web interface for the Configuration Panel. This interface provides the tools to create, configure, and administer the components of the integration of the Mobile Add-On.

System Monitor

The system monitor is the web interface for the Administration and Monitoring Portal (Admin Portal). This monitoring portal is provided to view mobile users' synchronization activities, including fetches, transactions, and pushes.

1.3 Mobile Exchange Persistent Layer

The mobile exchange persistent layer allows the transfer of data from SAP to the mobile client as well as the transfer of data from the mobile client to SAP. This exchange process provides a consistent way to manage and capture mobile client-related master and transaction data changes within SAP. The mobile exchange persistent layer uses the SAP NetWeaver enhancement framework to implement change detection triggers.



1.4 Delta Detection Routine

The delta detection routines are the triggers within the enhancement framework that detect modifications to data of concern to the mobile application. When changes are made to production data in the back end system, the delta detection routines will capture the change event. If a given change is one that will affect the data in the mobile application, then the routine will update the appropriate exchange table in the mobile exchange persistent layer. Intelligence is incorporated in the delta detection routines to determine if a change is one of concern to the mobile application. If it is not, then the change is ignored by the routine.

1.5 Change Detection Configuration Set

The change detection configuration set controls how the change detection should be carried out by the mobile exchange persistent layer. Customer-defined configurations are protected through a reserved customer namespace, such as "Y*" or "Z*". The change detection configuration set is mobile application-independent. However, it is a set of mobile application-specific change detection rules. The rules are configurable through the Configuration Panel.

1.6 Data Object Handler Class Repository

The data object class handler repository links BAPI wrappers with mobile data objects and encapsulates all business logic related to mobile applications. Internally standard BAPIs or custom logic can be implemented

to provide optimal mobile support. The object-oriented design provides benefits such as reliability, reusability, extensibility and maintainability.

All transaction updates are performed through standard BAPIs in order to ensure data integrity. The repository also supports streamlining multiple SAP transactions or multiple BAPIs for improved business processes. The data objects within the repository are application-dependent and delivered in the relevant package based on The SAP Business Suite Application hierarchy, such as /SMERP/MM or /SMERP/PM for ERP, or /SMCRM/MM or /SMCRM/PM for CRM.

The mobile data objects contained within the repository support result-set field selection, data filtering and security checks as defined by the integration rules. The mobile data objects also support monitoring and logging applications. Mobile data objects are configurable through the Configuration Panel. Different rules can be defined for every class handler for each mobile application.

1.7 Mobile Integration Configuration Set

The Mobile Integration Configuration Set within the business logic layer contains the user interface components presented in the Configuration Panel. The tools within this configuration set allow for the creation, configuration, and administration of the components within the business logic layer.

1.8 Application Authentication Services

Application authentication services allow you to perform additional mobile-related authentications that are not available using the standard SAP authentications and security profiles. You can configure these additional authorizations from the Security Settings screen in the Configuration Panel.

1.9 BAPI Wrapper

The BAPI wrapper layer provides a consistent way to expose business logic and data from the SAP system to external mobile applications. The BAPI wrappers are decoupled from the business logic in SAP, making them easy to reconfigure or customize.

The standard BAPI wrapper library consists of a collection of custom developed BAPIs based on the SAP integration framework. BAPI wrappers improve interface consistency with standard naming conventions and development standards. The BAPI wrapper library is application-dependent and delivered in the relevant package based on the SAP Business Suite Application hierarchy, such as /SMERP/MM or SMERP/PM for ERP, or /SMCRM/MM or /SMCRM/PM for CRM. BAPI wrappers set parameters for mobile data objects to post and pull data in the SAP system. When the Server calls to SAP, these calls are encapsulated in a BAPI wrapper to ensure that SAP responds in a consistent manner and performs the actions that SAP is asking for.

The Configuration Panel is used to configure BAPI wrappers by assigning them to mobile data objects and handler methods (GET, CREATE, UPDATE, or DELETE), function groups, and packages within SAP.

1.10 System Monitor

Administration and monitoring activities that take place behind the SAP Java connector for the SAP are performed through the System Administration and Monitoring Panel. The Administration Panel is a problem-detection tool utilizing SAP's standard application log database. It allows administrators to trace who is logging into the mobile applications, what work they're doing, and the data being transferred between the mobile devices and the SAP system. The Administration Panel speeds problem resolution using real-time data and traceability.

1.11 Configuration Module

The Configuration Module is the SAP configuration Web interface panel. The Configuration Panel simplifies modifications to existing mobile applications. It adds a layer of business logic for filtering at a system or user level. Data is mapped to SAP and pulled from SAP modules based on the user's credentials in the SAP system. Therefore, administrators can allow remote workers to only navigate and see data relevant to their job location or function.

1.12 Working with Push Scenarios

A push scenario pushes emergency work orders to the corresponding recipients. Following are the basic steps to follow a push instance from generation in SAP to reception on the client.

1. The push exchange process initiates the push trigger based on the push conditions. Conditions are defined as filter rules in the push exchange object. For instance, **work order priority = 1** is considered an emergency work order in the base product release.
2. The work order that satisfies the push conditions inserts a record into the push register table `/SYCLO/PSH01` with an object key as the work order number and a push status of NEW.
3. The event `/SYCLO/BACKGROUND_JOB_EVENT` is raised after the work order is saved, which triggers the background job for the push processor agent.
4. The push processor job `/SYCLO/CORE_PUSH_PROG_PROG` is triggered, either by using the event or the time frequency. This trigger is based on specific customer processes.
5. The push processor determines the recipients for the push work order and builds the data for each recipient as a separate instance. The instance is stored in the outbound message queue `/SYCLO/PSH02` with queue ID = PUSH, using the staging database.
6. The push instance displays one of the following statuses, viewable in the push monitor in the Administration Panel:
 - NEW
 - PROCESS
 - CANCEL
 - COMPLETED

- SRV_COMP
7. The Agentry application within the SAP Mobile server calls the push BAPI `/SYCLO/PM_DOPUSHWORKORDER_GET` for every predefined time interval and checks the push queue for new items.
 8. The Agentry application within the SAP Mobile server sends the push data to the respective clients depending on the user credentials that match the push instance.
 9. Once the client receives the push message, it sends the client confirmation back to the Server and the Server calls the BAPI `/SYCLO/CORE_PUSH_STAT_UPD` to update the confirmation with status `CLNT_CONF` back to SAP.

1.13 Outbound Trigger Handling

An outbound trigger allows a mobile application to interface with external systems such as the Agentry application within the SAP Mobile Server from SAP. Outbound triggers can be integrated into standard mobile application processes, such as push processing. You can define different types of outbound triggers, such as an HTTP trigger, file trigger, Web service trigger, and so on. You define an outbound trigger definition in the IMG activity.

Outbound triggers are configured for each mobile application. Therefore, the mobile application must be defined first. The Outbound trigger handler must be developed before it can be assigned to a trigger. An outbound trigger handler should implement interface `/SYCLO/IF_CORE_OUTB_TRIGGER` and should be a subclass of `/SYCLO/CL_CORE_OTRIG_BASE`.

2 Overall Synchronization Processing

The xChange data model is a process where information is gathered about data on the client compared to the data available on the back end and when it was last modified in either place. With this information, an implementor can configure a synchronization process so that the only information retrieved is information that has been modified since the last time the client synchronized with the server. It is recommended that the xChange data model be implemented whenever possible as it is almost always a more efficient model of synchronization, retrieving a smaller subset of the data stored on the client during any given transmit.

Synchronization With xChange Data

Synchronization using the xChange data model is the preferred configuration method for most objects. Using this method, the xChange object uses EFI triggers to track data changes. During synchronization of the mobile client with SAP, the mobile data object uses the exchange table maintained by the xChange object to determine the changes considered important. The data retrieved and sent to the client reflects all changes and additions since the mobile client's last transmit. Unchanged data is not retrieved again, as it already exists on the client and does not need to be replaced or updated.

Synchronization Without xChange Data

If synchronization is configured without using xChange data, no changes are tracked between transmits and the change detection layer is not involved. During synchronization of the mobile client, the mobile data object does not use the xChange table, but rather retrieves all data the client should contain, regardless of whether or not it has changed since the last transmit.

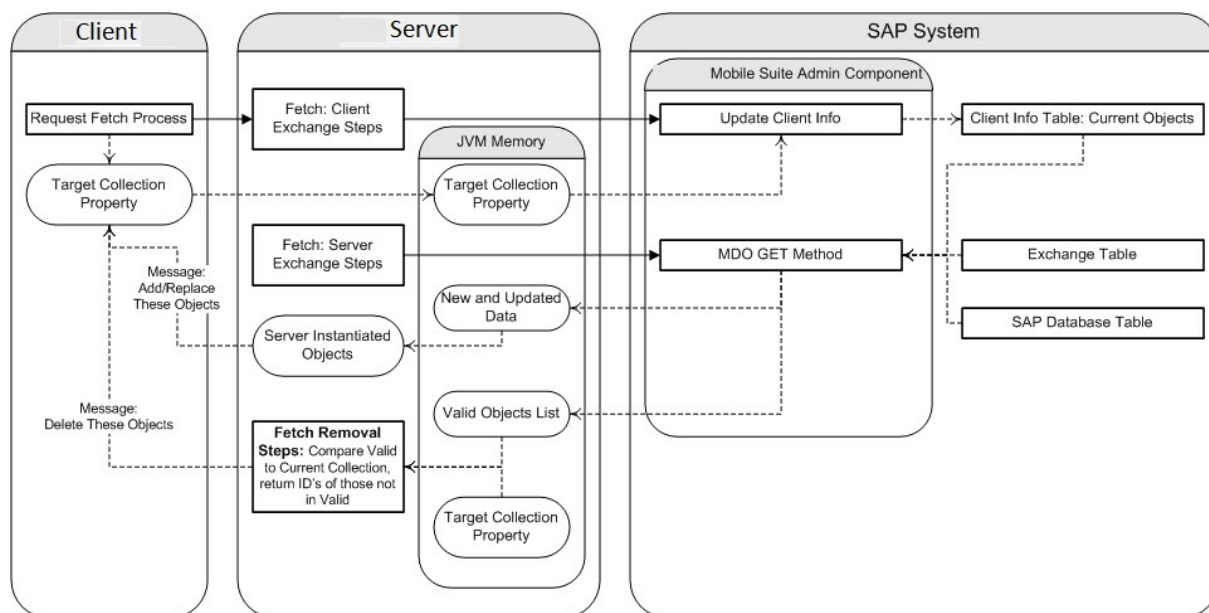
One example for not employing the xChange data model is if a majority of an object collection's records are changed between transmits. In this case, it may make sense to configure the object to synchronize without using the xChange data model. Time is saved by not comparing records through the xChange table and updating the entire collection at each transmit.

Another example is the download of backend customization settings to the mobile client. Customization settings are typically a small volume and do not require xChange support.

2.1 Fetch Synchronization

A fetch defines how the SAP Mobile Platform server synchronizes data for a target object collection. This object collection must be a top-level collection within the module. A fetch is made up of steps that retrieve the data for the collection from the back end system. These steps are grouped into three categories within the Fetch definition: Client Exchange Steps, Server Exchange Steps, and Removal Steps. A fetch can also include properties to store data captured from the user and validation rules for those property values.

Whenever possible, the synchronization processing of an object collection through a fetch definition should use the exchange data model. This allows for the retrieval of only those objects in need of addition or replacement in the collection targeted by the fetch, as well as the return of the key property values of only those objects in need of removal. In a typical environment, such objects are a small subset of the total number of objects stored in the collection. Using the exchange data model significantly reduces the amount of data retrieved during a given transmit from the SAP client to the SAP server.



1. The client initiates a transmit.
2. A request is sent to the SAP server to process a fetch targeting the object collection property.
3. The fetch client exchange steps call through the Java connector and through the BAPI wrapper in order to update the client information table with the key property and the last update timestamp for each object in the collection targeted by the fetch.
4. The fetch server exchange steps call the [GET](#) method in the mobile data object for the business object (for example, [Workorder](#), [Serviceorder](#), [Material](#)).
5. The mobile data object [GET](#) method references the xChange object if exchange processing is being used. If xChange processing is used, the xChange object provides the xChange table to the mobile data object.
6. The [GET](#) method reads the information from the client info table, the xChange table, and the SAP database table for the business object.
7. The [GET](#) method then compares the information returned from each of the tables to determine which objects need to be added or updated on the client. The [GET](#) method also generates the valid objects list, which is a list of all objects the client should currently have within the target collection property. Both the new and updated objects and the valid objects list are returned to the SAP server. This information is stored in the JVM memory.
8. The server instantiates the new objects based on the return data from the server xChange steps. These objects will be returned to the client to be added or replaced within the target collection property when the server-side fetch processing is complete.
9. The fetch removal steps compare the collection of objects from the client to the valid objects list. The key property of any objects found in the collection but not in the valid objects list are returned to the SAP server by the removal steps. These key properties are then sent to the client with messaging indicating these objects should be removed from the collection on the client.
10. The target collection on the client now contains up-to-date objects and object data.

2.2 Complex Table Synchronization

The complex table definition defines a table of records containing multiple fields stored on the SAP client in a structured and searchable format. A complex table can contain large amounts of data with records numbering in the thousands. Included in the complex table are the fields for its records and indexes on fields to provide search functionality and structure to the overall data in the table. The complex table definition also defines how its data is synchronized.

The synchronization of the complex table can include processing to only replace records whose data has changed in the SAP database, add new records, and return information about records to be removed. This processing is implemented using the xChange data model of synchronization. This method is recommended for all downstream synchronization as it is more efficient in most use cases, and is the standard method for synchronization.

The following description provides the general processing and data flow that occurs during synchronization of a complex table in a typical implementation. This processing assumes the use of the xChange data model of synchronization, which allows for incremental updates of data on the client.

→ Tip

For larger data sets, it may be necessary to configure streaming within the Mobile Add-On. Streaming will alter the processing logic from the method described here and is discussed in the next section.

1. The client initiates a transmit and makes a request to the server to synchronize a given complex table. Included in the request is the name of the complex table definition to synchronize and the date and time the data for the complex table was last updated.
2. The server processes the request, beginning by calling the constructor method for the complex table Java class that corresponds to the specific complex table definition. If the xChange data model is being used, the last update value from the client for the complex table is then updated to the client information table in the SAP database.
3. The server next calls the `build()` method within the complex table Java class. This method call ultimately results in the call to the `GET` method specified in the mobile data object for the complex table.
4. The `GET` method compares the last update timestamp in the client information table to the records in the xChange table as well as the SAP database for the complex table. Any records in the xChange table not marked for deletion and with a timestamp value newer than the last update table result in the corresponding record in the source table in the SAP database are retrieved.
5. The `GET` method next compares the last update timestamp in the client information table to the records in the exchange table that are marked for deletion. The key value is retrieved for any records marked for deletion and where the timestamp is newer than the last update value in the client information table.
6. The `GET` method returns both the list of new and updated records, and the list of key fields for records marked for deletion to the server. These values are cached in the Java Virtual Machine running for the server.
7. The `deleteIterator()` method is called by the server. This method retrieves the cached items from the list of key values for records to be deleted, returning these values to the server.
8. The server sends the list of key values to the client, instructing the client to delete the records with matching key field values from the complex table.
9. The `dataIterator()` method in the complex table Java class is called next by the server. This method returns the records from the cached new and updated records in the Java Virtual Machine memory. These records are returned to the server which uses the data to create records for the complex table being synchronized. The server sends the records to the client in chunks of 200 records at a time.

10. When the client receives a chunk of records, it matches them to the existing records within the complex table based on the key field. Any matching records on the client are replaced with the new ones. Any records in the new set that do not match existing records on the client are added to the complex table. The complex table's indexes are then updated to match the new data.
11. The client responds to the server that the records have been received and processed. The next chunk of records, if any, are sent to the client. This portion of the process repeats until all records for the complex table have been processed.
12. Once all new and updated records for the complex table have been downloaded, the next complex table is synchronized, repeating this entire process. Note that the order in which the complex tables for an application are synchronized is not guaranteed.

Complex Table Rebuild and Synchronization Without xChange Data

In the previous process, the synchronization steps for complex tables were discussed. In that process, it was assumed that the exchange data model is used, allowing for incremental updates to the records within the complex table as stored on the SAP client. Some complex tables within the application do not use this method of synchronization. For those complex tables, the synchronization process is a slightly different.

Also, there are situations when the complex table will be in a rebuild state, meaning that the complex table is to be completely replaced and rebuilt on the client. When in a rebuild state, the synchronization for the complex table does not use xChange data. Note that a complex table can be in a rebuilt state if the definition for it has been modified on the server, and also under other situations that can be controlled by the system administrator. The server checks the rebuild state of the complex table during each synchronization by calling the method `reload()`. This method returns true if the complex table is in a rebuild state.

When not using the xChange data model for synchronization, all records to be stored in the complex table on the client must be retrieved, rather than only retrieving those that have been modified or added in the back end system. In this case, the synchronization process is similar to the one previously described, but with the following differences:

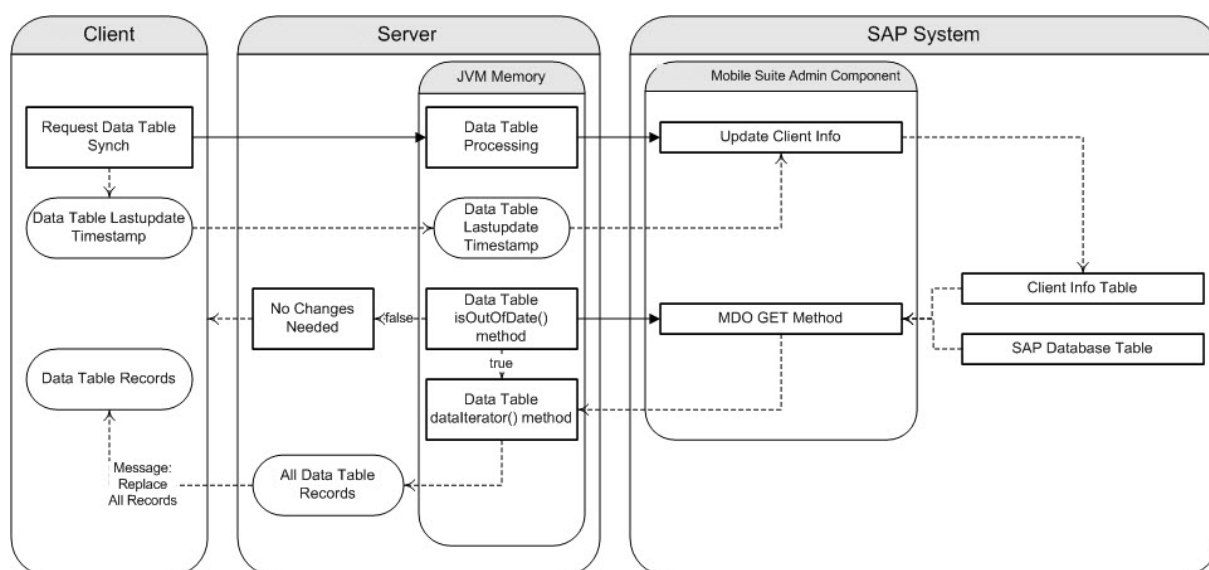
- The `GET` method within the mobile data object does not use the client information records or the xChange table. Both of these items are specific to the xChange data process and therefore are not a part of the logic when this method is not in use.
- The `GET` method, when called, returns all records that should be stored in the complex table in the client. As previously described, these records are cached within the Java Virtual Machine for the server.
- The data in this cache is still returned by the `dataIterator()` and processed in the same manner for download to the client. However, the `deletedIterator()` will not return records, as all records in the complex table are replaced with those returned during the latest synchronization. Since all data in the complex table is being replaced, it is not necessary to delete any records.

2.3 Data Table Synchronization

A data table definition defines a set of records stored on the client. Each record consists of two fields containing a key and value. A data table is intended to contain a small number of records (less than 100) that can be displayed to users in drop-down lists and other uses. A data table is defined at the application level and is available to all modules of the application. Its structure also defines how its data is synchronized.

Synchronization of a data table can include processing to determine whether or not the data table's records are out of date. The synchronization of a data table's records is an all or none operation, meaning either all of its records are replaced or none of them are replaced. Individual data table records cannot be replaced. The determination of whether or not a data table should be reloaded on a client device is based on the data table's last update time stamp, as stored on the client, compared to the date and time the source data was last modified in the back end system.

The following diagram illustrates the synchronization processing of a data table. This process assumes the xChange data model is in use. For data tables, the information exchange consists only of the date and time the data table's data was last downloaded to a given client, and the date and time when that data was last updated in the back end system.



1. The client initiates a transmit and makes a request to the server to synchronize the data in a data table. Included in this request is the name of the data table definition to synchronize and the date and time the data for the data table was last synchronized.
2. The server makes a call to the Java constructor method for the data table class. Next, as a part of the Java logic, the server calls into Mobile Add-On to provide the last update timestamp from the data table. This information is recorded in the client information table in the SAP database.
3. The server next calls the java method `isOutOfDate()` in the data table class. This method call results in a call to the mobile data object for the data table, specifically to the `GET` method for that MDO.
4. The `GET` method compares the last update time stamp for the data table from the client to the date and time when the source table in the database was last changed. If there have been no changes, the return to the `isOutOfDate()` method indicates this. This Java method then returns `false` to the server indicating there is no change needed. This response is sent to the client and the processing for the data table synchronization is complete. The client then makes its next synchronization request.
5. If the data in the database has changed, the records for the data table are retrieved. This includes all records the data table should contain, as this return data will completely replace the records currently stored on the client. These records are cached in the JVM memory for later reference. The `isOutOfDate()` Java method for the data table class returns `TRUE`. The `TRUE` return results in a call by the server to the data table Java method `dataIterator()`.
6. The `dataIterator()` method is called iteratively by the server to retrieve each of the cached records for the data table. The return of each call is a data table record to be sent to the client.

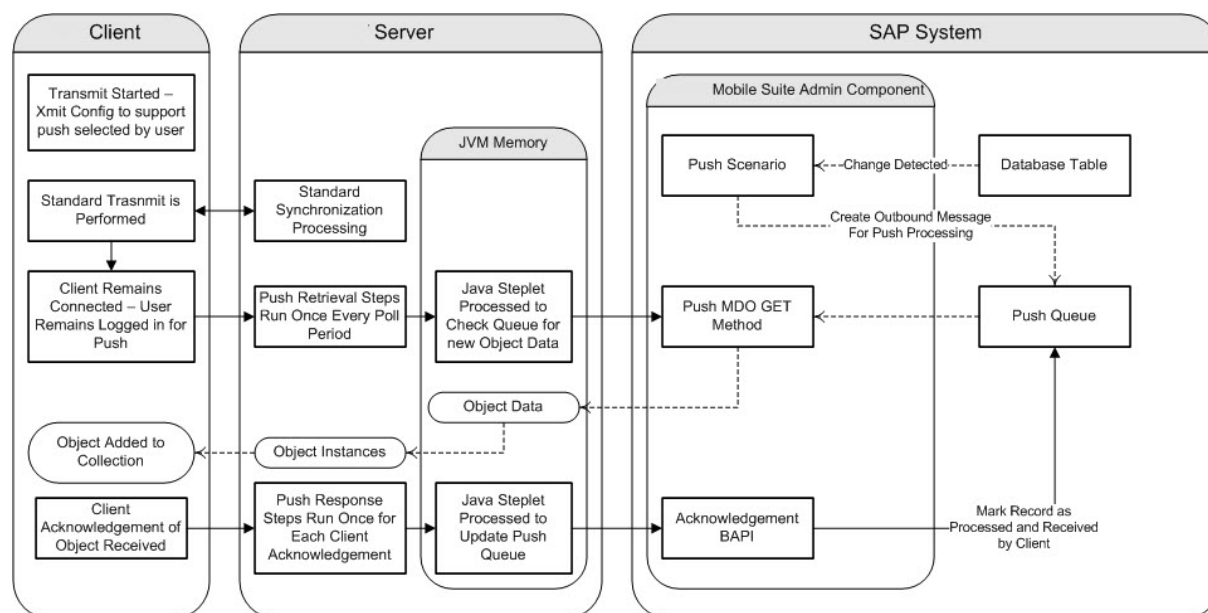
- Once all records have been retrieved, they are returned to the client. The client then replaces the current records in the data table with the records just received.
- If any additional data tables remain for synchronization, this process is repeated for each of them. Once all data tables have been synchronized, the transmit process moves to the next data definition to be synchronized.

2.4 Server Push Synchronization

A push defines when it is necessary to push an object from the back end system to the SAP client and how that object's data is retrieved. A push provides real-time data synchronization for server-to-client data transfer, targeting a top-level object collection property within the same module. The push determines if changes have been made to the back end system and also retrieves the needed data to send those changes to the client. Part of the push definition is the optional behavior to notify users when data has been pushed to their clients.

In a push scenario, data that fits the qualifications of the scenario defined in the Mobile Add-On is placed into the push queue in SAP. This data is then pushed to clients through a standard push process. Push scenarios are managed through the ConfigPanel and are used to connect a mobile data object with the appropriate xChange object for the push synchronization process.

The following diagram and subsequent descriptions provide the general processing and data flow that occurs during a push in a typical implementation. Push processing in SAP is a separate process from the push steps performed on the SAP server.



Push Scenario Processing

- A data object is changed in SAP, triggering the push process.

2. A push instance is instantiated and recorded in the push instance register with a status of **NEW** and a system push event is raised.
3. The push data set is calculated, the push recipient(s) are determined, and an outbound message is generated and ready for pickup in the push queue. The push data set is saved as data attached to the outbound message.
4. The outbound message is picked up by the server through its push/polling steps. Each server picks up the outbound messages only for mobile clients that stay connected to the server.
5. The outbound message reaches the end of its life cycle when the client sends a confirmation, the server sends a confirmation, or the push instance expiration time is reached.

SAP Server and Client Push Processing

1. The client connects to the network to perform a transmit and remains connected to the server.
2. The server begins the poll processing at the set interval.
3. The push retrieval steps call the **GET** method in the mobile data object for the push.
4. The **GET** method reads the information in the push queue in SAP and returns the object data to the method called as a push retrieval step.
5. The server instantiates the new objects retrieved by the push.
6. The server pushes down the objects one at a time to the appropriate client based on the user information collected by the push retrieval step.
7. The client sends an acknowledgement to the server indicating an object has been received. The server processes the push response step, which calls the BAPI acknowledgement to update the confirmation status of the message in the push queue in the SAP database.

2.5 Transaction Synchronization

The transaction definition defines data to be captured on the SAP client. As a part of its definition, the transaction includes a target object type, data values to be captured, client-side data validation, and updating its data to the back end system by the SAP server during synchronization. Transactions can add new object instances, edit an existing object, delete an object, or modify a complex table or data table record. Each of these behaviors is exhibited by a different transaction type, selected during the creation of the transaction.

Processing a transaction's data during synchronization involves the transaction and all its property values being sent from the client to the server. The server then processes and executes that transaction's server data state and server update steps, which are child definitions to the transaction. Data state steps are provided to support collision detection and collision handling and are rarely used in the SAP application. Server update steps, or simply update steps, perform the actual processing of the transaction's data and updating or adding it to the SAP database. The server data state and server update step definitions are both step usage definitions, meaning they use step definitions within the same module as the transaction.

Within the Mobile Add-On, the mobile data object (MDO) for the business object is used to perform the processing. The step definitions processed as update steps for the transaction contain classes that inherit from the Agency Java API class `Steplet`. The method `doSteplet()` is called by the server during the processing of the `Steplet` class to perform the actual step processing.

Depending on the type of transaction being processed, the method within the MDO being called will differ. Within the MDO there are methods for *GET* (not typically used by transactions), *CREATE*, *UPDATE*, and *DELETE*. The last three of these are typically used in transaction processing according to the following:

- *Add* transaction - *CREATE* method
- *Edit* transaction - *UPDATE* method
- *Delete* transaction - *UPDATE* or *DELETE* method

For *Delete* transactions, the object instance targeted by the transaction is removed from the client. However, it is rare that the data is also removed from the SAP database. Typically, the processing of the *Delete* transaction is handled by an update method in the MDO. This processing may include actions such as changing the status of the item (for example, from *STARTED* to *COMPLETE*), or updating an item as deprecated, or some other similar change.

3 Using the Mobile Data Objects (MDOs)

A mobile data object (MDO) represents a mobile combined view of data and activity for an SAP business object. Mobile data objects are data repositories in the namespace that can get, create, update, and delete information in SAP. They encapsulate the business logic of mobile applications by defining transactions, data structures, and business rules.

There are four types of mobile data objects:

- Data Table (DT) - a simple representation of SAP business objects KEY and VALUE (see the topic [Data Tables Overview \[page 27\]](#) for more information)
- Complex Table (CT) - a two-dimensional representation of a business object with a single table of multiple columns (see the topic [Complex Tables Overview \[page 30\]](#) for more information)
- Standard Data Object (DO) - a multi-dimensional representation of a business object with multiple tables representing different subsets of the business object (see the topic [Standard Data Objects Overview \[page 40\]](#) for more information)
- Composite MDO (CM) - a multi-dimensional, multi-table representation of an aggregation of two or more MDOs.

You can use the ConfigPanel to easily modify mobile data object properties such as object type, class handlers, data filters, and other settings. For example, instead of modifying BAPIs to change what information is retrieved from SAP and pushed out to mobile devices, you can use the ConfigPanel to modify mobile data objects and set up data filter rules.

3.1 BAPI Wrappers

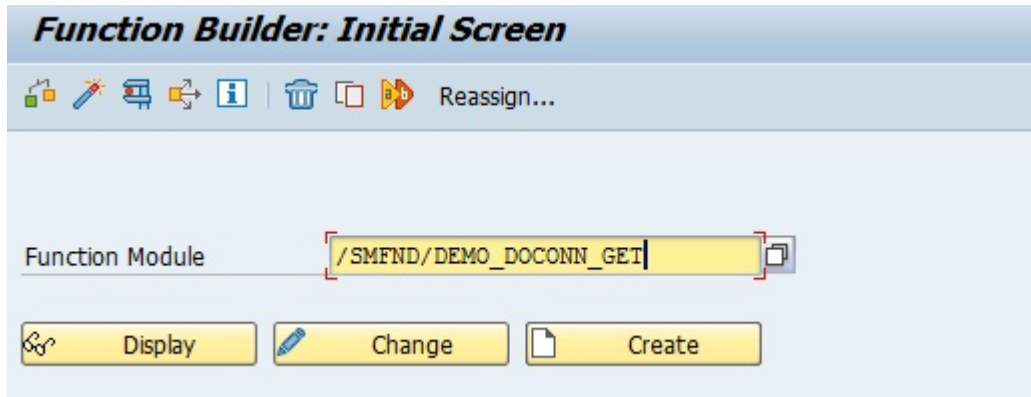
MDO methods are currently exposed to the Agentry mobile application through function modules. Therefore, a single **CRUD** method is mapped to a single Agentry BAPI.

Agentry integration BAPI wrappers are assigned to mobile data objects. Access the [BAPI Wrapper Configuration](#) page from the home page of the ConfigPanel.

Executing BAPI Wrappers Stand-alone

You can develop and test the MDO and BAPI coding and configuration before the mobile application is available. Take the following steps to create and test a BAPI.

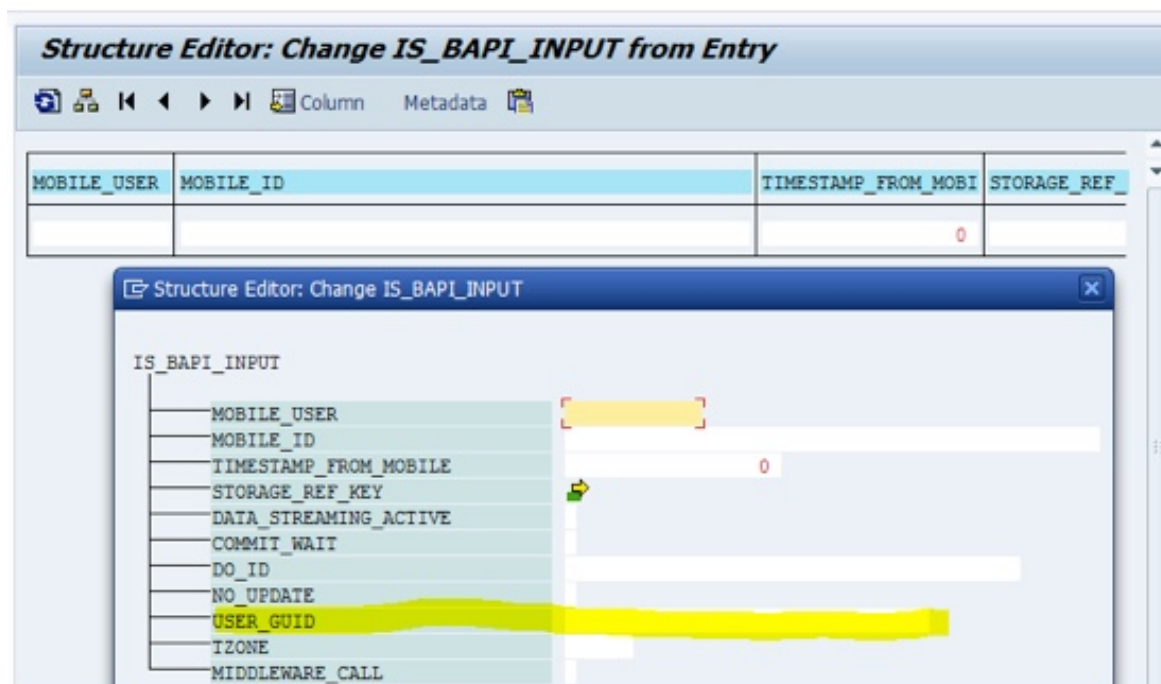
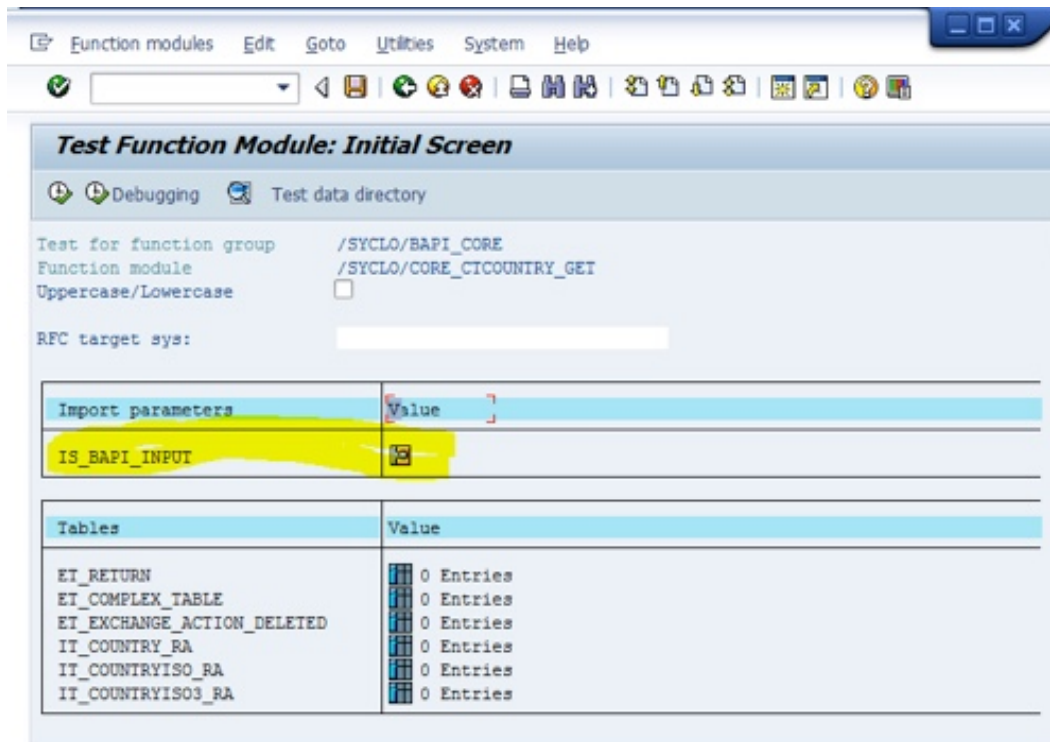
1. Use transaction code **SE37** and press **F8** to execute the BAPI and the underlying mobile data object method.



2. Emulate the mobile user. Every Agency BAPI has an importing structure `IS_BAPI_INPUT`.

ⓘ Note

The field `USER_GUID` is required. It identifies the mobile user connecting to the backend. Without a valid `USER_GUID`, the BAPI returns no data. Other input may be required, depending on which BAPI is being tested.



- Run transaction code /n/smfnd/myuser to locate the *USER_GUID*. If you change the user name, you can locate the *USER_GUID* for other users.

User GUID	Mobile Application	User	Lock Flag	App Block	Mobile Application Description	Release	Chng By	Changed Time Stamp
09F65531013FE5BE10000000A441E47	SAP_FLIGHT_MANAGER_10				SAP Flight Manager Demo 1.0.0	1.0.0		20,160,316,195,716
798EF85345F93B5AE10000000A441E47	ZSAP_FLIGHT_MANAGER_10				ZSAP Flight Manager Demo 1.0.0	1.0.0	II	20,150,714,230,236
DDEBFD53973B3A5AE10000000A441E47	YSAP_FLIGHT_MANAGER_10				YSAP Flight Manager Demo 1.0.0	1.0.0		20,160,316,205,334

4. If no user data was found, do the following:
 1. Run transaction `/n/syco/admin` to access the mobile application Admin Portal.
 2. Select **Administration** > **User Management**.
 3. Click **Create** and enter your **SAP User Name**, **Mobile App**, and any string for the **License Number**. The license number is required but is unimportant during testing of the BAPI stand-alone, so you can use any string.
 4. Click **Save**. You can now search the table again by using the transaction code `/SYCLO/MDW00`.

3.2 Data Tables Overview

A data table (DT) is a simple representation of SAP business objects KEY and VALUE.

A data table is a table of simple key and value pairs.

One common example is a table for the days of the week.

Data Table Sample

Number	Name
1	Monday
2	Tuesday
3	Wednesday
4	Thursday
5	Friday
6	Saturday
7	Sunday

Adding an Existing Data Table to a New Mobile Application

All SAP business partners have a Type of Person, Organization, or Group. This functionality is already developed as a data table in the ABAP Class Handler `/Syclo/CL_CRMD_BP_CAT_D0`. To include this table in your mobile application, take the following steps.

1. From the ConfigPanel Home page, select the [Mobile Data Object Configuration](#) link.
2. Click [Create](#).
3. Complete the highlighted fields as shown in the following example:

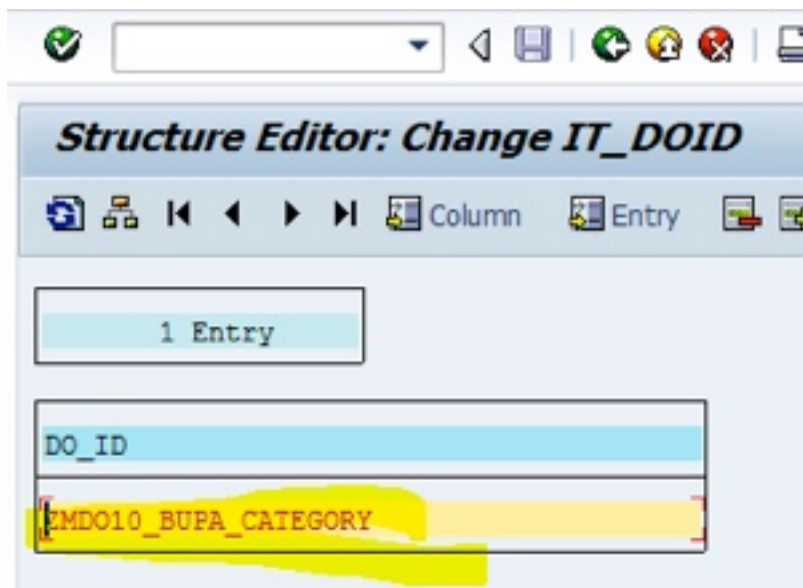
4. Click [Save](#) to save your entries.
5. A customizing request is required as your new configuration is recorded. If no test request exists, click [Create Request](#), otherwise click [Continue](#).
6. Set the Mobile Application Filter to [MDO examples](#) to see the new data table MDO.

7. You can now test the new Data Table MDO.

Executing the Single Data Table BAPI

Because data tables have to return key-value pairs, there is a common CORE BAPI called `/Syclo/CORE_DT_GET`. This BAPI has a special input table to specify the desired MDO IDs. To execute the BAPI, take the following steps.

1. Run transaction SE37, enter `/Syclo/CORE_DT_GET()` and press **F8** to execute.
2. In the `IS_BAPI_INPUT`, set the `USER_GUID` fetched in the topic [BAPI Wrappers \[page 24\]](#).
3. In the input table `IT_DOID`, enter your newly created data object ID, click the green button with the back arrow, and execute.



4. Review the `ET_DATA_TABLE` that results. There should be three rows.

The screenshot shows the SAP Structure Editor for the table `ET_DATA_TABLE`. The title bar reads "Structure Editor: Display ET_DATA_TABLE". Below the title bar, there is a toolbar with icons for navigation and editing. The main area shows a table with three rows: `DO_ID`, `KEY`, and `VALUE`. The rows are highlighted in yellow.

DO_ID	KEY	VALUE
ZMDO10_BUPA_CATEGORY	1	Person
ZMDO10_BUPA_CATEGORY	2	Organization
ZMDO10_BUPA_CATEGORY	3	Group

Data Tables Using a Data Filter

You can implement some special features using data tables, such as filtering the data returned. For example, you can filter on a shipping condition and set up two options; **ASAP** and **Standard**. To set up filtering, take the following steps:

1. Implement the method `GET_DATA_TABLE()`.
2. In the ConfigPanel, navigate to the [Data Filter](#) tab of the MDO `ZMDO10_SHIPCOND` and add the two values for the filter `SHIP_COND`.
3. Once you create your filter, ensure that it is handled correctly when selecting the data in the `GET_DATA_TABLE`. Therefore, employ the method `MAP_LOCAL_DOF_FILTER()`.

The method operates the object `me->oref_mdo_data`, which contains all of the data passed to the MDO via the ConfigPanel and BAPI signature.

4. Re-execute the BAPI `/SYCLO/CORE_DT_GET` with the `DO_ID` of `ZMDO10_SHIPCOND` to get the data for the mobile application.

Some considerations:

- One drawback is that the values of the shipping table must be known. You can provide the same data for the ConfigPanel implementing `GET_DATA_FILTER_VALUESSET()`. Check the ConfigPanel to see the values.
- You could also refactor the `GET_DATA_TABLE` method to not repeat the `SELECT` code. You can see all the helper methods in the class `/scyclo/cl_crmcore_valset_tools`.
- Consider the implementations of `/SYCLO/CL_CORE_DT_HANDLER` as a model to copy.

Data Tables Using Domain Values

Sometimes, the data table is the product of the domain values. For example, you can offer the days of the week as shown by the domain `WFM_DAY_OF_WEEK`. To implement days of the week, there is the handler `/SYCLO/CL_CORE_DT_DOMVAL_DO`, which is a subclass of `/SYCLO/CL_CORE_DT_HANDLER`. This handler allows you to develop the new data table through the ConfigPanel alone.

Implement the days of the week example using the following steps:

1. Create a new MDO called `ZMDO10_DAYS_OF_WEEK` in the ConfigPanel.
2. On the *Data Filter* tab, create a new filter for the `DDIC_DOMAIN_VALUE` that matches the domain name.
3. Execute the BAPI `/SYCLO/CORE_DT_GET` with the `DO_ID` of `ZMDO10_DAYS_OF_WEEK`.

3.3 Complex Tables Overview

A complex table (CT) is a two-dimensional representation of a business object with a single table of multiple columns.

Despite its name, a complex table is a flat table. It is considered "complex" only in relation to a data table. A data table contains a simple key/value pair, but a complex table can have multiple values defined for a single key.

From the ABAP side, the Get for complex tables acts much like that for data tables. In the case of complex tables, the resulting structures vary with each complex table MDO. To handle this additional complexity, there is the class handler `/SYCLO/CL_CORE_CT_HANDLER`.

Because the return structure varies, each complex table has its own BAPI function module.

All CRUD methods are supported, but as with data tables, the best practice is to only implement the GET method. The updates and creates are typically used with the standard data object MDOs.

A country code table is an example of a complex table.

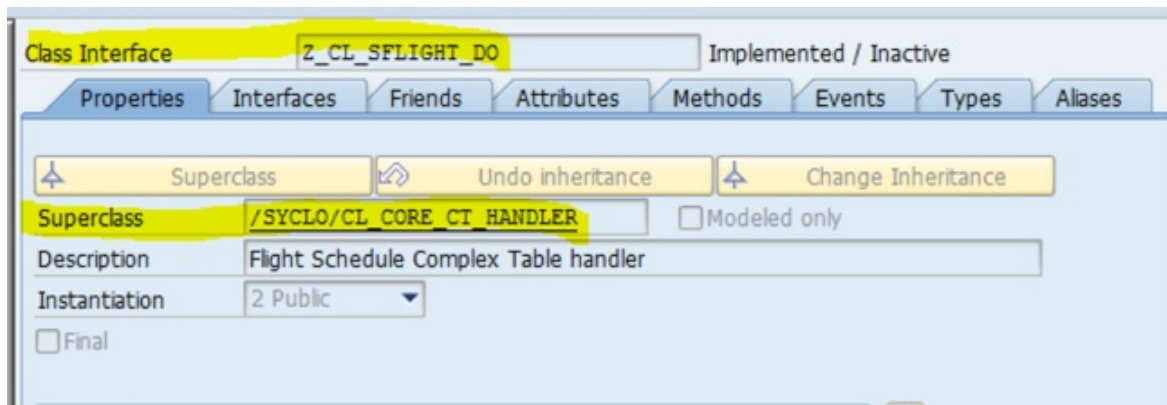
Complex Table Sample

Key	Name	Continent
DE	Germany	Europe
FR	France	Europe
CA	Canada	North America

Complex Tables - Basic Example

For this example, look at the flight model. Many of the tables are candidates for a complex table MDO. For example, the flight table (SFLIGHT) is a listing of scheduled flights. Take the following basic steps to create a sample complex table.

1. Using transaction code SE24, create a new subclass of `/SYCLO/CL_CORE_CT_HANDLER` called `Z_CL_SFLIGHT_DO`.



2. Implement the method `GET_COMPLEX_TABLE()`. Like `GET_DATA_TABLE()` for data tables, map the method as the *GET* of the MDO. At this point, the method is in a basic form, simply selecting all the data from `SPFLIGHT`.


```

METHOD get_complex_table.

    "The following structure contains reference to all supported output parameter
    "from MDO handler. Output parameter name is set to the same as what is declared
    "in receiving BAPI wrapper signature for simplicity.
    DATA:
        BEGIN OF ls_mdo_output_vals,
            et_complex_table TYPE REF TO flighttab,
        END OF ls_mdo_output_vals.

    "initialize output tables
    CREATE DATA: ls_mdo_output_vals-et_complex_table.

    "Perform main SQL selection
    SELECT * FROM sflight
        INTO CORRESPONDING FIELDS OF TABLE ls_mdo_output_vals-et_complex_table->*.

    " --return output data to MDO data object, which mapped to BAPI
    " Wrapper data container automatically by POSTPROCESS_MAPPING
    me->oref_mdo_data->set_mdo_output_via_ref_struct(
        EXPORTING is_mdo_output = ls_mdo_output_vals ).

ENDMETHOD.

```

- Using transaction code SE37, create an RFC-enabled function module for the flight schedule complex table *GET BAPI* and name it *Z_MDO_CTFLIGHT_GET*.
- Set the interface parameters. As with all Agency BAPIs, there are standard input and output structures, *IS_BAPI_INPUT*, and *ES_BAPI_OUTPUT*. There is also a standard table, *ET_RETURN*, for any return messages. The other standard table is *ET_EXCHANGE_ACTION_DELETED*.
- Each complex table GET BAPI has the same named output table that holds the results for the GET. It is called *ET_COMPLEX_TABLE*. However, the structure changes for each MDO, using a single line of code to include the standard MDO BAPI code.

```

Function module: Z_MDO_CTFLIGHT_GET (Active)
Source code:
1 FUNCTION z_mdo_ctflight_get .
2
3  *** Local Interface:
4  *** IMPORTING
5  ***   VALUE(IS_BAPI_INPUT) TYPE  /SYCLO/CORE_BAPI_INPUT_STR OPTIONAL
6  *** EXPORTING
7  ***   VALUE(ES_BAPI_OUTPUT) TYPE  /SYCLO/CORE_BAPI_OUTPUT_STR
8  *** TABLES
9  ***   ET_RETURN STRUCTURE  BAPIRET2 OPTIONAL
10 ***   ET_COMPLEX_TABLE STRUCTURE  SFLIGHT OPTIONAL
11 ***   ET_EXCHANGE_ACTION_DELETED STRUCTURE
12 ***   /SYCLO/CORE_CT_DELETED_STR OPTIONAL
13
14
15   INCLUDE /syclo/core_bapi_template_incl.
16
17 ENDFUNCTION.

```

- Using transaction code */n/syclo/configpanel*, access the MDO configuration and add a new complex table MDO called *ZMDO10_CTFLIGHT*, with parameters as shown in the following example:

Data Object Detail (Create Mode)

General Setting ResultSet Field Selection Data Filter Data Staging Proxy Setting

Basic Data

Mobile Data Object Id: * ZMDO10_CTSFLIGHT

Description: * Flight

Data Object Type: * Complex Table Mobile Application: * MDO examples

Reference Business Object:

Data Object Handler Settings

Data Object Handler: Z_CL_SFLIGHT_DO : Flight Schedule Complex Table Skip Exception Processing: ☐

Get Method: GET

Create Method:

Update Method:

Delete Method:

Exchange Object Settings

Exchange Object:

Conversion Exit Setting

Enable Conv. Exit Overwrite: ☐

Middleware Reference Info

Reference Middleware Object Type:

Activation

Data Object Active: ☒

7. Within the ConfigPanel, access the *BAPI Wrapper Configuration* and add a wrapper for the *Z_MDO_CTSPFLI_GET* BAPI. Assign the new BAPI wrapper to the *GET* method of the newly created MDO.

General Assignment

BAPI Wrapper Info

BAPI Wrapper Name: Z_MDO_CTSFLIGHT_GET

Description:

Mobile Data Object Assignment List

Add Delete

Mobile Application	Mobile Data Object Id	Description	Method Type	Active Flag	Default Assignment
ZMDO_EXAMPLES_10	ZMDO10_CTSFLIGHT	Flight	GET	☒	☒

Assignment Detail

Mobile Application: * MDO examples

Mobile Data Object Id: CT - Flight

Description: Flight

Method Type: Get method

Active Flag: ☒

8. Using transaction code SE37, test the Z_MDO_CTSFLIGHT_GET BAPI. Remember to set the USER_GUID to IS_BAPI_INPUT.

Structure Editor: Display ET_COMPLEX_TABLE

Column Entry Metadata

10 Entries

MAN	CAR	CONN	FILDATE	PRICE	CURRE	PLANETYPE	SEATSMAX	SEATSOCC	PAYMENTSUM	SEATSMAX_B	SEATSOCC_B	SEATSMAX_F	SEATSOCC_F
800	LH	0400	28.02.1995	899.00	DEM	A319	350	4	2,639.00	0	0	0	0
800	LH	0454	17.11.1995	1,499.00	DEM	A319	350	2	2,949.00	0	0	0	0
800	LH	0455	06.06.1995	1,090.00	USD	A319	220	1	1,499.00	0	0	0	0
800	LH	0455	31.12.1996	1,919.00	DEM	DC-10-10	380	190	344,479.69	0	0	0	0
800	LH	2402	21.08.1997	555.00	DEM	A319	300	100	0.00	0	0	0	0
800	LH	2402	22.08.1997	590.00	DEM	A330-300	300	250	0.00	0	0	0	0
800	LH	2402	25.08.1997	490.00	DEM	A330-300	300	290	0.00	0	0	0	0
800	LH	2402	30.08.1997	485.00	DEM	A330-300	300	290	0.00	0	0	0	0
800	LH	3577	28.04.1995	6,000.00	LIT	A319	220	1	600.00	0	0	0	0
800	SQ	0026	28.02.1995	849.00	DEM	DC-10-10	380	35	1,684.00	0	0	0	0

Complex Tables - Field Selection

Some database tables have hundreds of fields, many of which you do not need to display through the mobile application. To improve performance, you can ignore the unneeded fields. To ignore unnecessary fields, take the following steps:

1. Implement the method `GET_FIELD_SELECTOR_TABLES()` and specify which tables you are selecting from. Additionally, you can also specify a field.

```

METHOD /syclo/if_core_fldsel_serv~get_field_selector_tables.

FIELD-SYMBOLS:
  <selector_table> TYPE LINE OF /syclo/core_fldsel_serv_tab,
  <req_fld>         TYPE LINE OF /syclo/core_req_fldlist_tab,
  <excl_d_fld>      TYPE LINE OF /syclo/core_excl_d_fldlist_tab.

super->get_field_selector_tables( ).

REFRESH: et_fldsel_table_list,
         et_req_field_list,
         et_exclusion_field_list.

* Field selector for method GET
APPEND INITIAL LINE TO et_fldsel_table_list ASSIGNING <selector_tabl
<selector_table>-do_handler = me->clname.
<selector_table>-do_mthd = 'GET'.
<selector_table>-tabname = 'SFLIGHT'.

* Required fields for method GET
APPEND INITIAL LINE TO et_req_field_list ASSIGNING <req_fld>.
<req_fld>-do_handler = me->clname.
<req_fld>-do_mthd    = 'GET'.
<req_fld>-tabname    = 'SFLIGHT'.
<req_fld>-fieldname  = 'CARRID'.

APPEND INITIAL LINE TO et_req_field_list ASSIGNING <req_fld>.
<req_fld>-do_handler = me->clname.
<req_fld>-do_mthd    = 'GET'.
<req_fld>-tabname    = 'SFLIGHT'.
<req_fld>-fieldname  = 'CONNID'.

- ENDMETHOD.

```

2. Update the `GET_COMPLEX_TABLE` method to pay attention to the new customization.

```

METHOD get_complex_table.
"
The following structure contains reference to all supported output parameter
from MDO handler. Output parameter name is set to the same as what is declared
in receiving BAPI wrapper signature for simplicity.
DATA:
  BEGIN OF ls_mdo_output_vals,
    et_complex_table TYPE REF TO flighttab,
  END OF ls_mdo_output_vals.

*Variables
DATA: lv_select_clause TYPE string.

"initialize output tables
CREATE DATA: ls_mdo_output_vals-et_complex_table.

"Determine if there is a field catalog associated with this get method
lv_select_clause = me->build_field_selector_string( iv_mthd = 'GET' ).

"Perform main SQL selection
SELECT (lv_select_clause) FROM sflight
  INTO CORRESPONDING FIELDS OF TABLE ls_mdo_output_vals-et_complex_table->.

```

3. In the ConfigPanel, click the [MDO Configuration](#) link. Navigate to the [ResultSet Field Selection](#) tab of the MDO `ZMDO10_CTSFLIGHT` and specify the desired fields.

General Setting **ResultSet Field Selection** Data Filter Data Staging Proxy Setting

Handler Info

Mobile Data Object Id: **ZMDO10_CTSFLIGHT** Description: Flight

Data Object Handler: **Z_CL_SFLIGHT_DO** Get Method: GET

Field Selection Detail

Search:

Field Catalog	Field Active	Field Description	Data Format
▼ Handler Method - GET	☒		
▼ Table - SFLIGHT	☒	Flight	
• Field - CARRID	☒	Airline	CHAR 3
• Field - CONNO	☒	Connection Number	NUMC 4
• Field - CURRENCY	☒	Airline local currency	CUKY 5
• Field - FLDATE	☒	Flight Date	DATS 8
• Field - PAYMENTSUM	☐	Booking total	CURR 17.2
• Field - PLANETYPE	☒	Plane Type	CHAR 10
• Field - PRICE	☒	Airfare	CURR 15.2
• Field - SEATSMAX	☒	Maximum capacity economy class	INT4 10
• Field - SEATSMAX_B	☐	Maximum capacity business class	INT4 10
• Field - SEATSMAX_F	☐	Maximum capacity in first class	INT4 10
• Field - SEATSOCC	☐	Occupied seats economy class	INT4 10
• Field - SEATSOCC_B	☐	Occupied seats business class	INT4 10
• Field - SEATSOCC_F	☐	Occupied seats in first class	INT4 10

Sort Options

☒ By Field Name

☐ By Field Description

☐ By DDIC Sequence

4. Execute transaction code SE37 and test the BAPI `Z_MDO_CTSFLIGHT_GET`. Notice that fewer values (columns) are returned.

MAN	CAR	CONN	FLDATE	PRICE	CURRE	PLANETYPE	SEATSMAX	SEATSOCC	PAYMENTSUM	SEATSMAX_B	SEATSOCC_B	SEATSMAX_F	SEATSOCC_F
LH	0400	28.02.1995		899.00	DEM	A319	350	0	0.00	0	0	0	0
LH	0454	17.11.1995		1,499.00	DEM	A319	350	0	0.00	0	0	0	0
LH	0455	06.06.1995		1,090.00	USD	A319	220	0	0.00	0	0	0	0
LH	0455	31.12.1996		1,919.00	DEM	DC-10-10	380	0	0.00	0	0	0	0
LH	2402	21.08.1997		555.00	DEM	A319	300	0	0.00	0	0	0	0
LH	2402	22.08.1997		590.00	DEM	A330-300	300	0	0.00	0	0	0	0
LH	2402	25.08.1997		490.00	DEM	A330-300	300	0	0.00	0	0	0	0
LH	2402	30.08.1997		485.00	DEM	A330-300	300	0	0.00	0	0	0	0
LH	3577	28.04.1995		6,000.00	LIT	A319	220	0	0.00	0	0	0	0
SQ	0026	28.02.1995		849.00	DEM	DC-10-10	380	0	0.00	0	0	0	0

Complex Tables - MDO Filter and BAPI Inputs

There are two basic ways to influence which rows are returned:

- Set up a filter on the MDO object

- Offer input parameters in the BAPI

To add MDO and BAPI input filters, take the following steps:

1. For the MDO filter, implement the method [GET_DATA_FILTER_LIST](#). Specify the method, the name of the filter, the reference table, and reference field.
2. In the ConfigPanel, click the [MDO Configuration](#) link on the home page. Navigate to the [Data Filter](#) tab of the MDO [ZMDO10_CTSFLIGHT](#) and set a filter (for example, [planetype](#)).
3. For the BAPI input filter, you can input new parameters to the BAPI interface. By convention and for flexibility, define these filters as range tables. Add new parameters using transaction code SE37.
4. Process the new filters in the SQL statement in the [GET_COMPLEX_TABLE\(\)](#) method.
Note the two methods, [map_local_dof_filter\(\)](#) and [map_local_mobile_filter\(\)](#). Convert data from the ConfigPanel and BAPI, respectively, into the local data structure. The data structure is then incorporated in the [WHERE](#) clause of the [SELECT](#).

GET_COMPLEX_TABLE	Active
-------------------	--------

```

    "in filter service method GET DATA FILTER_LIST
DATA: BEGIN OF ls_dof_filter_vals,
    carrier      TYPE REF TO /syclo/core_range_tab,
    planetype    TYPE REF TO /syclo/core_range_tab,
END OF ls_dof_filter_vals.

* "The following structure contains reference to all supported import
* "parameters supported by MDO handler. Parameter name is set to the same
* "as what is defined in BAPI wrapper signature for simplicity.
DATA: BEGIN OF ls_mdo_input_vals,
    it_carrier_ra TYPE REF TO /syclo/core_range_tab,
END OF ls_mdo_input_vals.

"The following structure contains reference to all supported output parameter
"from MDO handler. Output parameter name is set to the same as what is declared
"in receiving BAPI wrapper signature for simplicity.
DATA: BEGIN OF ls_mdo_output_vals,
    et_complex_table TYPE REF TO flighttab,
END OF ls_mdo_output_vals.

*Variables
DATA: lv_select_clause TYPE string.

" -->Retrieve filter settings as defined via ConfigPanel.
" ConfigPanel filter settings has been mapped to MDO data object
" by INITIALIZE_MDO_DATA
me->oref_mdo_data->map_local_dof_filter(
    CHANGING cs_filters = ls_dof_filter_vals ).

" --Retrieve supplied MDO input data and map to local variables.
" MDO input data are supplied by BAPI wrapper and mapped to MDO
" data object by PREPROCESS_MAPPING
me->oref_mdo_data->map_local_mobile_filter(
    CHANGING cs_filters = ls_mdo_input_vals ).

"initialize output tables
CREATE DATA: ls_mdo_output_vals-et_complex_table.

"Determine if there is a field catalog associated with this get method
lv_select_clause = me->build_field_selector_string( iv_mthd = 'GET' ).

"Perform main SQL selection
SELECT (lv_select_clause) FROM sflight
    INTO CORRESPONDING FIELDS OF TABLE ls_mdo_output_vals-et_complex_table->*
    WHERE carrid IN ls_dof_filter_vals-carrier->* AND
           carrid IN ls_mdo_input_vals-it_carrier_ra->* AND
           planetype IN ls_dof_filter_vals-planetype->*.
  
```

5. Execute transaction code SE37 and test the BAPI `Z_MDO_CTSFLIGHT_GET`. Observe that only flights with plane type containing `DC` are returned.
6. Set the `IT_CARRIER_RA` to `LH`. Notice that only the matching flight is returned.

ⓘ Note

If *Sign* is blank it is assumed to be `I` for inclusive, and if *Option* is blank, it is assumed to be `EQ` for equals.

Complex Tables - Considerations

Look to the implementation of [/SYCLO/CL_CORE_CT_HANDLER](#) as a model to copy.

3.4 Standard Data Objects Overview

A standard data object is a multidimensional representation of a business object with multiple tables representing different subsets of the business object.

A standard data object (DO) is a collection of objects that can be represented in multiple tables.

An example use of a standard data object would be for sales orders.

Standard Data Object Sample

Order#	Customer	Order Date
1001	Acme	01-Jan-2015
1002	Media Store	04-Feb-2015

Order#	Line#	Product	Qty
1001	01	Widget-1	100
1001	02	Accessory-1	5
1002	01	Component-14	17

Standard data Objects - GET Method Example

The GET method for standard data objects supports multiple output tables. The data tables and complex tables provide special handling to help you in developing the GET method. The data table supports a set single return table of set structure (key/value). The complex table supports a single return table of varying structures. The Standard Data Object provides more flexibility.

An example use case is to implement the GET method for airline booking as a standard data object. In addition to the booking (table: SBOOK), you can provide the flight information in SFLIGHT. To implement the GET method for airline booking, follow these steps:

1. Set the output tables.
 1. Using transaction code SE24, create a new subclass of the base [/SYCLO/CL_CORE_DO_HANDLER_BASE](#) and name it [Z_CL_SBOOK_DO](#).
 2. Implement the GET method. The same concepts apply from the [Complex Tables Overview \[page 30\]](#) and [Data Tables Overview \[page 27\]](#), except the output structures are expanded. Start with the initial SQL and output tables, and parameters on the BAPI. Note the method [set_mdo_output_via_ref_struct\(\)](#).

3. Match the output tables with the parameters on the BAPI.
 4. In the ConfigPanel, go to [MDO Configuration](#) and create an MDO of type [Standard Data Table](#).
 5. In the ConfigPanel, go to [BAPI Wrapper Configuration](#) and map the new MDO to the BAPI.
 6. Use transaction code SE37 to test the BAPI. Remember to use the previously determined [USER_GUID](#) in the [IS_BAPI_INPUT](#).
2. Select output fields. Some database tables have hundreds of fields, many of which are not displayed on the mobile device. To improve performance, you can ignore the unneeded fields.
 1. Implement the method [GET_FIELD_SELECTOR_TABLES\(\)](#) and specify which tables are being selected from. Additionally, specify fields needed from subsequent SELECTS as required (optional).
 2. Update the GET() method to pay attention to the new customization.
 3. In the ConfigPanel, access the [MDO Configuration](#). Navigate to the [ResultSet Field Selection](#) tab of the MDO [ZMDO10_DOSBOOK](#) and specify the desired fields.
 3. Set MDO filters and BAPI parameters. There are two basic ways to influence which rows are returned: set up a filter on the MDO object or offer input parameters in the BAPI.
 1. For the MDO filter, implement the method [GET_DATA_FILTER_LIST](#). Specify the method, the name of the filter, the reference table, and reference field.
 2. In the ConfigPanel, access the [MDO Configuration](#). Navigate to the [Data Filter](#) tab of the MDO [ZMDO10_DOSBOOK](#) and set a filter (for example, set a filter for the flight date as [fldate](#)). Instead of specifying static values, the [DOF Rule](#) is set to [Syclo Filter Class](#). We supply the generic data handler [/SYCLO/CL_CORE_DATE_DOF](#) through the DOF rule. With the generic data handler, we specify the following parameter:
[&TYPE=CURR_DATE&FROM_OFFSET=6500&TO_OFFSET=90](#).
 This example, when implemented, calculates a date range from 6500 days in the past and 90 days into the future.
 3. Input new parameters to the BAPI interface for the BAPI input filter. By convention and for flexibility, define these filters as range tables. Add new parameters using transaction code SE37.
 4. Process the BAPI input filters in the SQL statement in the [GET_COMPLEX_TABLE\(\)](#) method.. The two methods are [map_local_dof_filter\(\)](#) and [map_local_mobile_filter\(\)](#). The methods convert data from the ConfigPanel and BAPI, respectively, into the local data structure. The local data structure is then incorporated in the [WHERE](#) clause of the [SELECT](#).

ⓘ Note

The BAPI parameter [IT_PLANETYPE_RA](#) corresponds to a plane type, which does not appear on the booking table [SBOOK](#). Because the BAPI parameter does not appear, there exists a two-step process of SQL selects: first determine which keys are relevant, and then fill out the requested data for these keys.

5. Execute transaction code SE37 and test the BAPI [Z_MDO_DOSBOOK_GET](#). Observe that fewer rows are returned based on the [FLDATE](#) MDO filter than with no filtering. Re-execute with an additional filter on [IT_PLANETYPE_RA](#). Notice that the additional filter added yields even fewer rows.
4. An even more critical way to influence which rows are returned is take advantage of an *Exchange* enabled object. The concept is to only retrieve records that have changed (have been added) since the last synchronization (transmit) between the mobile app and the back end. Use the following steps to define an exchange enabled object.

ⓘ Note

You can also apply this exchange behavior to a complex table.

Understand how to develop an EFI, exchange tables, and exchange objects to support an exchange object in the GET method.

5. Set the middleware object registry. You can extend the exchange concept through the middleware object registry, further influencing the data returned by the GET() method. The Middle Object Type name is freeform, but the SAP Mobile Platform server must know what it is. When a user transmits, all of the keys of the object currently on the mobile device are stored to a middleware table for this type.

The GET() method determines the keys where data needs to be fetched. Then we compare this list, the full list of valid keys, and the objects currently on the device.

If there is a key that is not part of the exchange and not on the device, the data must still be fetched. The described scenario can happen if the object was cleared from the device.

If there is a key that is part of the exchange, but a version of the data is newer on the device, there is no need to refetch the data.

1. In the ConfigPanel, access the [MDO Configuration](#) to set the [Reference Middleware Object Type](#).
2. Update the SQL results by the exchange in the GET() method.

Creating, Updating, and Deleting Standard Data Objects

Although data tables and complex tables do support Create, Update, and Delete, these methods are developed for the standard data objects.

For the retrieving data, we tend to SELECT the data directly from database tables, mostly for performance reasons. However, for Create, Update, and Delete routines, we search for an API that SAP provides us. There is no practical or safe way to try to reimplement the business logic.

One downside of reading directly from tables while using APIs is that it can lead to some confusion between field names and field formats. Be aware that these differences do exist.

Note

The process for Update and Delete for SDOs are analogous to Create.

1. Find an appropriate API. This step requires some research. For Flight Booking, there is the SAP function module [BRF_FLIGHT_BOOKING_ADD_SINGLE\(\)](#) for creating a new flight reservation.
2. Using transaction code SE24, implement the [/SYCLO/IF_CORE_DO_HANDLER-CREATE\(\)](#) method. Set up the input and output structures that match the BAPI. Initialize the input structure retrieving the data from the BAPI. Call the API function module. Map the output structure with any results.
3. Using transaction code SE37, create the create BAPI [Z_MDO_DOBOKING_CRT](#). Aside from the standard input and output structure and a return message table, inputs and outputs to match the CREATE() method.
4. Make the appropriate ConfigPanel changes. Aside from the standard input and output structure and a return message table, modify the inputs and outputs to match the CREATE() method.
5. Using transaction code SE37, test the BAPI.

4 Change Detection

In the context of mobile applications, especially offline-capable applications, the ability to detect data changes in the backend SAP system is critical.

For offline-capable mobile applications, a fat mobile client is used. All categories of data from backend SAP system are downloaded to the mobile client in order to support continuous usage of the application in a completely disconnected mode. With the advancement of mobile technology, data being downloaded to the mobile device can be quite large. When the mobile application client reconnects to the backend system, it needs to be synchronized with the backend system in order to receive the latest information. A complete refresh of the mobile client is typically not practical from the perspective of economical user experience and functionality. Instead, the ability to perform incremental synchronization with the backend system is required. In order to support incremental synchronization from mobile client, change detection support in the backend system is mandatory.

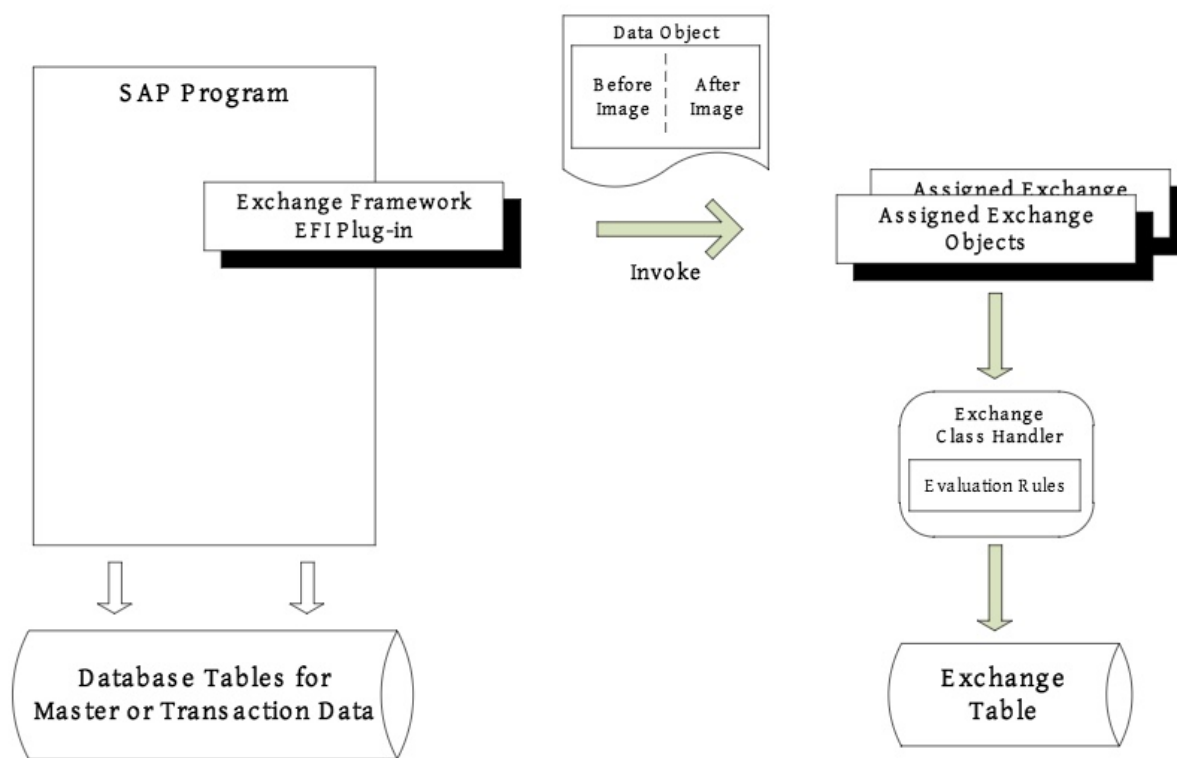
The xChange Framework that is part of the SAP Framework takes advantage of the different technologies available within the SAP system. It provides a unified and consistent approach to implementing change detection in the ABAP based AP Business Suite systems, and support information consumers such as offline-enabled Agency metadata-based mobile applications. Non-Agency metadata-based applications such as oData-based mobile applications with delta time token support can also take advantage the xChange Framework for delta calculation.

4.1 xChange Framework Overview

The xchange framework consists of four key concepts:

- Exchange Framework Implementation (EFI)
- Exchange handler
- Exchange table
- Exchange object

The following diagram is a high-level diagram that illustrates the change detection approach within the xchange framework:



Change detection within the xchange framework is based on the following concepts:

- Data object change in SAP is handled by an SAP update program. The SAP update program can be in the form of a program, a function module, a class, an ABAP routine, and so on.
- **Exchange Framework Implementation (EFI):** The SAP update program always embeds EFI as a part of it. EFI serves as a listening agent of the standard SAP data update process. EFI collects and builds the before and after image of the data object being changed. The before and after image is based on the relational data model of the data object, in the form of tables or structures. EFI also determines whether the data object is being created, updated, or deleted. It provides the information through an action flag with the following values:

- I: Insert
- U: Update
- D: Delete

EFI then passes the before and after image of the data object, along with the action flag, to the exchange object. Actual change detection is performed by exchange object based on the logic and business rules defined for the exchange object.

- **Exchange Object:** Responsible for executing change detection business rules defined for the relevant mobile scenario. An exchange object is a logical configuration object that includes technical settings such as exchange handler, exchange table, or lock object. Exchange objects also include change detection business rules for the mobile scenario. Exchange objects are defined in the ConfigPanel of the SAP Framework. Exchange objects are always assigned to a single mobile application.
- **Exchange Table:** The SAP persistent database table where information identifying the changed data is stored. Typically, the Exchange Table uses a minimum standard structure which captures only minimum

information about the data object that has been changed, such as mobile application, object key, last changed time stamp, change action (Insert/Update/Delete), and user. For some situations, an extended form of Exchange Table can also be used in order to support additional performance or functionality requirement.

The xChange Framework allows the ability to extend the Exchange Table structure, as long as the minimum standard structure is implemented. It is recommended that separate Exchange Tables are defined for different types of SAP data object. The same Exchange Table is typically shared by multiple mobile applications which require change detection information for the same SAP data objects. By default, Last Changed Time stamp is captured in exchange table using the SAP system time zone.

The minimum standard structure for an Exchange Table is defined by the structure `' /SYCLO/CORE_EXCHANGE_KEY_STR '` (key field) and structure `' /SYCLO/CORE_EXCHANGE_SUB_STR '`. The following table lists the fields required as standard exchange table

Field Name	Data Element	Key Field?
MANDT	MANDT	Yes
OBJKEY	/SYCLO/CORE_EXCH_OBJKEY_DTE	Yes
MOBILE_APP	/SYCLO/CORE_MOBILE_APP_DTE	Yes
ACTION	/SYCLO/CORE_EXCH_ACTION_DTE	No
CHANGED_TS	/SYCLO/CORE_CHANGEDTS_DT	No
CHANGED_BY	/SYCLO/CORE_CHANGEBY_DTE	No
REC_STATUS	/SYCLO/CORE_EXCH_RECSTATUS_DTE	No

- Exchange Handler - The Exchange Handler is the technical ABAP Object class that contains the programming logic to support the change detection capability. A standard exchange handler base class `' /SYCLO/CL_CORE_EX_HANDLER_BASE '` is provided by the xChange Framework. The base class can be used for generic change detection. The generic change detection has the following behavior:
 - For an object with action flag 'I' - Insert, change detection returns Boolean value 'True'
 - For an object with action flag 'D' - Delete, change detection returns Boolean value 'True'
 - For an object with action flag 'U' - Update, all of the before/after image of the relational data tables or structures will be checked for changes. Value changes to any individual attribute of these tables or structures will cause change detection to return Boolean value 'True'
 - If change detection has a result of Boolean value 'True', the Exchange Table will be updated.

You should create a new exchange handler by declaring a subclass of `' /SYCLO/CL_CORE_EX_HANDLER_BASE '` in order to provide added enhancement to the standard generic change detection. Sub-classed exchange handler is a commonly used practice, as it gives you the flexibility to implement more targeted change detection support. Common use cases of using new exchange handler include:

- Refine the specific scope of before/after image detection that is supported. Scope is limited to what the EFI will provide, but can be more restrictive if required. This is done by redefine method `'GET_FIELD_SELECTOR_TABLES'`. This enhancement also allows you the ability to define individual

field by field change detection options via ConfigPanel settings that is not available in generic change detection

- Define condition filters for change detection. Condition filters allows you restrict the change detection to subset of SAP data object. For example, an order category condition filter can be defined for a work order exchange handler. This will allow xChange Framework to apply change detection logic to only maintenance work orders but not production orders, even though both share the same technical update program in SAP ERP system. This is done by refine method 'GET_DATA_FILTER_LIST'. Once declared, condition filters will be visible in ConfigPanel, and condition rules can be defined to control how change detection should be performed based on data set value.
- EFI/Exchange Object Assignment - By assigning an Exchange Object to an EFI, change detection for the Exchange Object is activated. When a SAP data object is changed, EFI is triggered. Each Exchange Object that is assigned to this EFI is invoked automatically. Same set of Before/After image from EFI is provided to each Exchange Object for it to perform the required change detection rules and logic. It is common when multiple applications use the same SAP data object, multiple Exchange Objects from different mobile applications are assigned to the same EFI. It is also common to have multiple Exchange Objects configured in the same mobile application and assigned to the same EFI. This allows the ability to define different change detection rules for the same technical object but different scenarios. For example, regular change detection for maintenance work order might use a different change detection rule than emergency work order with push support.

4.2 EFI Development

EFI Trigger Spot Determination

To determine the proper EFI trigger spot, analyze the standard SAP data object update process. You can identify and implement a trigger spot in different ways: User exit, BAdI implementation, explicit or implicit enhancement framework implementation, business transaction event, and so on. Given the key role of EFI to construct the before and after image data pair list of the SAP data object being changed, the EFI trigger spot determines the extent and computes the change direction.

If the complete before and after image data pair list can be provided by the EFI, full change detection is possible. If only a partial before and after image data pair list is provided by EFI trigger, it is not possible for the exchange object to detect changes for all possible data change variations. Sometimes, it might also be necessary to have multiple EFI trigger spots for the same SAP data object change to support comprehensive change detection for the data object

EFI Coding

Current xChange framework implementation requires that an ABAP include file is created. The include file can contain the programming logic for before and after image construction. Use transaction code SE38 to create the EFI include file.

Standard coding patterns have been established for EFI programming. The following pseudo-code pattern can be used for any EFI programming. Implement sections highlighted with a green background for the specific EFI.

```

*****
* Data Declaration Section
*****

*OO Reference Variables
DATA: lref_exception TYPE REF TO cx_root.

*Tables & Structures
DATA: lt_efi_data TYPE /syclo/core_exch_chngdata_tab,
      ls_efi_data LIKE LINE OF lt_efi_data.

*Variables
DATA: lv_efi_objkey TYPE /syclo/core_exch_objkey_dte,
      lv_efi_action TYPE /syclo/core_exch_action_dte,
      lv_efi_active TYPE wdy_boolean.

*Constants
CONSTANTS: lc_efi_incl_name TYPE /syclo/core_efi_incl_dte VALUE 'xxxx',
            lc_source_routine_name TYPE string VALUE 'yyyy'.

*****
* Main Section
*****

TRY.

    /syclo/cl_core_exch_serv=>logtrace_exch_begin( iv_efi_include = lc_efi_incl_name
                                                iv_source = lc_source_routine_name ).

    /syclo/cl_core_exch_serv=>get_exchange_active( lc_efi_incl_name ).
    IF lv_efi_active = abap_true.

        "Determine the proper EFI action.
        "Insert: use value /syclo/cl_core_constants=>exch_action_insert
        "Update: use value /syclo/cl_core_constants=>exch_action_update
        "Delete: use value /syclo/cl_core_constants=>exch_action_delete

        lv_efi_action = ???

        "Log EFI action
        /syclo/cl_core_exch_serv=>logtrace_exch_action_info( iv_action = lv_efi_action
                                                            iv_source = lc_source_routine_name ).

        "Determine the EFI object key

```



```
lv_efi_objkey = ???
```

```
"Build before/after image array
```

```
ls_efi_data-tabname = 'xxx'.
```

```
ls_efi_data-dref_new_data = ???
```

```
ls_efi_data-dref_old_data = ???
```

```
append ls_efi_data to lt_efi_data.
```

```
/sycto/cl_core_exch_serv=>update_exchange(
```

```
    EXPORTING iv_objkey = lv_efi_objke
```

```
            iv_efi_include = lc_efi_incl_name
```

```
            iv_action = lv_efi_action
```

```
            it_data = lt_efi_data ).
```

```
ENDIF.
```

```
CATCH cx_root INTO lref_sycto_exception.
```

```
/sycto/cl_core_appl_logger=>logger->catch_class_exception(
```

```
    EXPORTING iref_exception = lref_exception ).
```

```
ENDTRY.
```

4.3 Exchange Table Development

- To define a standard exchange table, use transaction code SE11 to create a transparent table in the data dictionary. Include the standard structures [/SYCLO/CORE_EXCHANGE_KEY_STR](#) and [/SYCLO/CORE_EXCHANGE_SUB_STR](#). Set structure [/SYCLO/CORE_EXCHANGE_KEY_STR](#) as key.

Field	Key	Ini...	Data element	Data T...	Length	Deci...	Short Description
MANDT	☒	☒	MANDT	CLNT	3	0	Client
.INCLUDE	☒	☒	/SYCLO/CORE_EXCHANGE_KEY_STR	STRU	0	0	Standard Exchange Table Key Structure
.INCLUDE	☐	☐	/SYCLO/CORE_EXCHANGE_SUB_STR	STRU	0	0	Standard Exchange Table sub-structure

- To define an exchange table with extended structure, in addition to the minimum standard structure for exchange table, add additional fields for the exchange table. For example, an extended exchange table with the additional fields of [WERKS](#) and [LGORT](#) in the following example:

Field	Key	Ini...	Data element	Data T...	Length	Deci...	Short Description	Group
MANDT	☒	☒	MANDT	CLNT	3	0	Client	
.INCLUDE	☒	☒	/SYCLO/CORE_EXCHANGE_KEY_STR	STRU	0	0	Standard Exchange Table Key Structure	
WERKS	☐	☐	WERKS_D	CHAR	4	0	Plant	
LGORT	☐	☐	LGORT_D	CHAR	4	0	Storage Location	
.INCLUDE	☐	☐	/SYCLO/CORE_EXCHANGE_SUB_STR	STRU	0	0	Standard Exchange Table sub-structure	

For extended exchange tables, redefine the exchange handler method [SAVE_EXCHANGE_TABLE](#) to update the additional attributes.

- The exchange table is not intended to be a history table; purge its table contents regularly. The standard purge program [/SYCLO/CORE_EXCH_PURGE_PROG](#) is provided to purge exchange tables. A background job to run the purge program periodically is recommended. The exchange object setting [Days to Keep History](#) controls the number of days of change history that are maintained in an exchange table..

- For optimal runtime performance, secondary indexes are recommended when defining a new exchange table.

4.4 Lock Object Development

Define a lock object for the exchange table during implementation of the xChange framework.

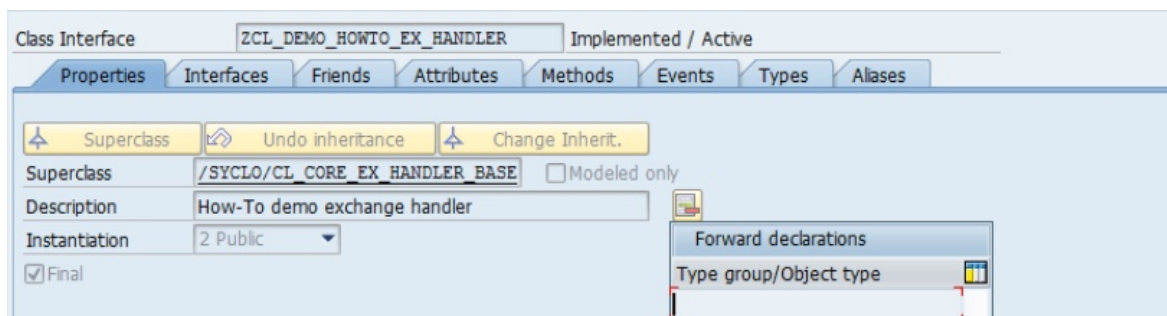
Define a lock object for the exchange table using transaction code SE11. A lock table is necessary to provide the necessary lock mechanism during the update process for the exchange table. Define the lock object information in the exchange object definition. If defined, it is checked when the exchange table is being updated.

4.5 Exchange Handler Development

An exchange handler contains the program code that executes the change detection logic algorithm based on the before and after image provided by EFI trigger.

There are seven key steps to developing the exchange handler:

- Create a custom exchange handler. Use transaction code SE24 to create a subclass handler using base class `/SYCLO/CL_CORE_EX_HANDLER_BASE`.



- Redefine the method `GET_FIELD_SELECTOR_TABLES` of the exchange handler, to declare a list of data pairs the exchange handler can support. The declared data pair list can be the full list of before and after images provided by the EFI triggers, or it can be a subset. Any data pair declared by the exchange handler, but not provided by EFI trigger is ignored. The data pair name defined in `/SYCLO/CORE-EX_FLDSEL_SERV_STR-TABNAME` should match the value defined in the EFI trigger before and after image data pair list, and it should be a valid DDIC table name or structure name. See the example code that follows on how to declare the data pair. Both table `BUT000` and `BUT050` are declared by the sample code. Change the sections highlighted with a green background for your specific exchange handler and data object.

```

*****
* Data Declaration Section
*****

*Tables & Structures

DATA: lt_field_selector_tables TYPE /syclo/core_ex fldsel_serv_tab.


*Field Symbols

FIELD-SYMBOLS: <selector_table> TYPE LINE OF /syclo/core_ex fldsel_serv_tab.


*****

* Main Section
*****

REFRESH et_fldsel_table_list.

APPEND INITIAL LINE TO lt_field_selector_tables ASSIGNING <selector_table>.

<selector_table>-ex_handler = me->clsname.

<selector_table>-tabname = 'BUT000'.

APPEND INITIAL LINE TO lt_field_selector_tables ASSIGNING <selector_table>.

<selector_table>-ex_handler = me->clsname.

<selector_table>-tabname = 'BUT050'.


et_fldsel_table_list = lt_field_selector_tables.

```

The declared data pair list is displayed on the ConfigPanel. With data pair list declared, individual table field level change detection settings can be defined through ConfigPanel. In the following screen shot, table *BUT000* and table *BUT050* are displayed.

Exchange Object Detail (Display Mode)

Create Copy Delete Change

Technical Settings Change Detection Field Selection Change Detection Condition Filter Linkage Settings Push Settings

Exchange Object Info

Exchange Object: ZHOWTO_DEMO_EXCHANGE_OBJECT Exch. Object Desc.: Z - HowTo Demo Exchange Object

ExchObject Handler: ZCL_DEMO_HOWTO_EX_HANDLER

Exchange Object Field Selector

Search

Field Catalog	Active Flag	Short Description
Table - BUT000	☒	BP: General data I
Field - ADDRCOMM	☒	Address Number
Field - AUGRP	☒	Authorization Group
Field - BIRTHDT	☒	Date of birth
Field - BIRTHPL	☒	Birthplace
Field - BPEXT	☒	External BP number
Field - BPKIND	☒	BP type
Field - BP_EEW_DUMMY	☒	Dummy
Field - BU_GROUP	☒	Grouping
Field - BU_LANGU	☒	Language
Field - BU_LOGSYS	☒	Logical system
Field - BU_SORT1	☒	Search Term 1
Field - BU_SORT2	☒	Search Term 2
Field - CHDAT	☒	Changed on
Field - CHILDREN	☒	
Field - CHTIM	☒	Changed at
Field - CHUSR	☒	Changed by
Field - CLIENT	☒	Client
Field - CNDSC	☒	Ctry of origin
Field - CNTAX	☒	

Selection Proposal

Get Proposal

Sort Options

☒ By Field Name
☐ By Field Description
☐ By DDIC Sequence

- Redefine the `GET_DATA_FILTER_LIST` method of the exchange handler to declare a list of change condition filters the exchange handler supports. A change condition filter requires a reference table or structure name, and a field name. The reference table or structure name should be one of the data pairs declared by the `GET_FIELD_SELECTOR_TABLES` method. If not, it is ignored.

Once you declare the filters, define the change condition rules in the ConfigPanel. The condition rules are evaluated as part of the change detection process. If the data object that is evaluated does not fulfill the change conditions, the change condition process is skipped, and a FALSE Boolean value is returned.

The standard evaluation algorithm is supported by the `/SYCLO/CL_CORE_EX_HANDLER_BASE` base exchange handler, `PROCESS_FILTER` method. The algorithm applies an *all or nothing* protocol. If a condition filter rule is defined for a line item table, all records for the data object line item table must fulfill the condition rule. If an *any or nothing* algorithm is required for line item level filters, you will need to redefine the `PROCESS_FILTER` method with your own program code.

In the following code example, two condition filters are declared: the `BU_TYPE` filter for the `BUT000` table, `TYPE` field, and the `ROLE_TYPE` filter for the `BUT100` table, `RLTYP` field. Change the section highlighted with a green background for your specific exchange handler and data object.

```

*Tables & Structures
DATA:
  lt_data_filter TYPE /syclo/core_ex_filter_serv_tab.

*Field Symbols
FIELD-SYMBOLS:
  <data_filter> TYPE LINE OF /syclo/core_ex_filter_serv_tab.

*****

* Main Section
*****

  REFRESH et_data_filters.

*Field Filter for exchange object
  APPEND INITIAL LINE TO lt_data_filter ASSIGNING <data_filter>.
  <data_filter>-ex_handler = me->clsname.
  <data_filter>-dof_name    = 'BU_TYPE'.
  <data_filter>-usage_tabname = 'BUT000'.
  <data_filter>-usage_fieldname = 'TYPE'.

  APPEND INITIAL LINE TO lt_data_filter ASSIGNING <data_filter>.
  <data_filter>-ex_handler = me->clsname.
  <data_filter>-dof_name    = 'ROLE_TYPE'.
  <data_filter>-usage_tabname = 'BUT100'.
  <data_filter>-usage_fieldname = 'RLTYP'.

  et_data_filters = lt_data_filter.

```

- Redefine the `GET_DATA_FILTER_VALUESSET` method of the exchange handler, to provide a predefined value list for specific condition filters. Condition filters with a predefined value list are displayed as dropdown list fields on the ConfigPanel. A condition filter that references a table field that has a domain value check list defined in DDIC is automatically shown as a dropdown list.

In the following code example, the specific value set for the `ROLE_TYPE` filter is constructed by the program logic. There is already a DDIC check value list for the reference table field `TYPE` of table `BUT000`. Both condition filters are displayed as a dropdown list on the ConfigPanel. Change the section highlighted with a green background for your specific exchange handler and data object.

```

*****

* Data Declaration Section

*****

*Tables & Structures

DATA: lt_filter_valset TYPE /syclo/core_ex_fltsrv_vset_tab.

*****

* Main Section

*****

REFRESH et_filter_valset.

* Business Partner Types

CLEAR: lt_filter_valset.

/syclo/cl_crmcore_valset_tools=>get_crm_bupa_roletype_valset(

    EXPORTING iv_ex_handler = me->clsname

               iv_do_mthd   = 'GET'

               iv_dof_name  = 'ROLE_TYPE'

    IMPORTING et_ex_valset  = lt_filter_valset ).

APPEND LINES OF lt_filter_valset TO et_filter_valset.

SORT et_filter_valset BY ex_handler dof_name value.

```


The ConfigPanel shows the dropdown list for both declared condition filters:

- Redefine the [PROCESS_FILTER](#) method of the exchange handler, to overwrite the standard condition filter of the rule evaluation algorithm (optional). For example, you have declared condition filters at the line item level and need to implement logic to support *any or nothing* condition filter rules logic.
- Redefine the [SAVE_EXCHANGE_TABLE](#) method of the exchange handler, to overwrite the standard update logic for exchange table (optional). For example, you have an extended exchange table structure, which includes additional nonstandard attributes in the exchange table that need to be populated.
- Implement the [PROCESS_SYNC_LINKAGE](#) method of the exchange handler, if linkage processing is going to be supported by the exchange handler.

4.6 Exchange Object Configuration

An exchange object is a customization object that combines technical objects such as an exchange class handler, exchange table, and a lock object with configuration rules.

The exchange object allows you to control the runtime change detection behavior by defining specific customizing rules for different mobile scenario use cases. Change detection functionality defined by each exchange object is limited to what the underline technical objects (example: exchange class handler, exchange table) can support.

You can create an exchange object through the ConfigPanel (► [IMG](#) ► [Agentry SAP Framework Configuration](#) ► [Exchange Process Setting](#) ► [Define Exchange Object Settings](#) ►).

Important attributes include the exchange object ID, description, mobile application, application area, exchange table name, exchange handler, and lock object. Use the [Active Flag](#) field to control whether the exchange object is enabled. Use the [Days To Keep History](#) field to control the archive and purge setting for exchange table content.

Agentry SAP Framework Configuration

Welcome B34209 | System - CRM(B00) | Today - 20.05.2013

[ConfigPanel Home](#)

Exchange Object By Application Area ►

- ▼ Cross-Application
 - ZHOWTO_DEMO_EXCHANGE_OBJECT
 - Sales

Exchange Object Detail (Display Mode)

Create Copy Delete Change

Technical Settings Change Detection Field Selection Change Detection Condition Filter Linkage Settings Push Settings

Exchange Object: ZHOWTO_DEMO_EXCHANGE_OBJECT Exch. Object Desc.: Z - HowTo Demo Exchange Object

Mobile Application: NW Gateway POC - Delta Services

Application Area: Cross-Application

Reference Business Object:

Exchange Table Name: ZHOWTO_DEMO_EX_T Exch. Table Desc.: Z - HowTo Demo Exchange Table

Exchange Lock Object:

ExchObject Handler: ZCL_DEMO_HOWTO_EX_HANDLER No Exchange Table Update: ☐

Active Flag: ☒ Days To Keep History: 180

Administrative Info

Based on the exchange handler selected, the [Change Detection Field Selection](#) tab shows a list of tables or structures that support individual field level change detection. In the following example, change detection only applies to the value change for the [MC_NAME1](#) and [MC_NAME2](#) fields of the Business Partner General Data 1 table ([BUT000](#)).

Exchange Object Detail (Change Mode)

Save
Cancel

Technical Settings
Change Detection Field Selection
Change Detection Condition Filter
Linkage Settings
Push Settings

Exchange Object Info

Exchange Object: ZHOWTO_DEMO_EXCHANGE_OBJECT
Exch. Object Desc.: Z - HowTo Demo Exchange Object

ExchObject Handler: ZCL_DEMO_HOWTO_EX_HANDLER

Exchange Object Field Selector

Search

Field Catalog	Active Flag	Short Description
Field - INITIALS	☐	Initials
Field - JOBGR	☐	Occupation
Field - LANGU_CORR	☐	Correspondence lang.
Field - LEGAL_ENTRY	☐	Legal Form
Field - LEGAL_ORG	☐	Legal Entity
Field - LIQUID_DAT	☐	Liquidation date
Field - LOCATION_1	☐	Int. Location No. 1
Field - LOCATION_2	☐	Int. Location No. 2
Field - LOCATION_3	☐	Check digit
Field - MARST	☐	Marital status
Field - MC_NAME1	☒	Name 1/Last Name
Field - MC_NAME2	☒	Name 2/First Name
Field - MEM_HOUSE	☐	
Field - NAMCOUNTRY	☐	Country for format
Field - NAME1_TEXT	☐	Full Name
Field - NAMEFORMAT	☐	Format name
Field - NAMEMIDDLE	☐	Middle Name
Field - NAME_FIRST	☐	First Name
Field - NAME_GRP1	☐	Name 1
Field - NAME_GRP2	☐	Name 2

Selection Proposal

Get Proposal

Sort Options

☒ By Field Name
☐ By Field Description
☐ By DDIC Sequence

Based on the exchange handler selected, the *Change Detection Condition Filter* tab shows list of condition filters supported. You can define condition filter rules in range table format. Use the *Active Flag* field of the *Rule Editor* area to enable or disable a rule. Use the *Ignore Data Creation*, *Ignore Data Deletion*, and *Ignore Data Update* fields to control whether change detection should be restricted to a specific data change process (Create / Update / Delete).

Exchange Object Detail (Change Mode)

Save Cancel

Technical Settings Change Detection Field Selection **Change Detection Condition Filter** Linkage Settings Push Settings

Exchange Object Info

Exchange Object: ZHOWTO_DEMO_EXCHANGE_OBJECT Exch. Object Desc.: Z - HowTo Demo Exchange Object

ExchObject Handler: ZCL_DEMO_HOWTO_EX_HANDLER

Exception Settings

Ignore Data Creation: ☐ Ignore Data Deletion: ☐ Ignore Data Update: ☐

Exchange Object Filter Rule Definition

Defined Filters

- Filter - BU_TYPE
- Filter - ROLE_TYPE

Rule Editor

Filter Name: BU_TYPE

Reference Table Name: BUT000 Reference Field Name: TYPE

Data Filter Rule Key: ZHOWTO_DEMO_EXCHANGE_OBJECT.ZCL_DEMO_HOWTO_EX_HANDLER.BU_TYPE

Enter Range Value

Sign: inclusive Option: =

Low Value: 3 - Group

High Value:

Active Flag: ☒

Rule List

Rule No.	Rule Type	Rule Value	Active Flag
00001	RANGE		☐

Create Delete

4.7 EFI Configuration

Configuration for the EFI trigger allows you to assign exchange objects to a specific EFI trigger.

To create a new EFI setting, use the ConfigPanel (► [IMG](#) ► [Agency SAP Framework Configuration](#) ► [Exchange Process Settings](#) ► [Define Enhancement Implementation Include \(EFI\) Settings](#) ►).

An EFI trigger is identified in the ConfigPanel by the EFI include file. You can assign a number of exchange objects for each EFI trigger. Multiple mobile applications can share the same SAP data object and require change detection support. It is also possible to define a different exchange object to support different change detection rules for multiple use cases within the same mobile application.

For example, an exchange object used to initiate a push process might have a different change detection rule than a regular change detection scenario. You can define a separate push relevant exchange object to support the push process.

An EFI include file must exist in the SAP system.

EFI Assignment Detail (Display Mode)

Create | Copy | Delete | Change

General | Assignment

Enhancement Implementation Include File Info

EFI Include Name:

Description:

Package:

You can assign multiple exchange objects to the same EFI trigger. Use the [Active Flag](#) field to enable or disable an assignment.

EFI Assignment Detail (Display Mode)

Create | Copy | Delete | Change

General | Assignment

EFI Include Name:

Description:

Package:

EFI Assignment List

EFI Include Name	Mobile Application	Exchange Object	Exch. Object Desc.	Active Flag
/SYCLO/CRMMD_EFI_PARTNER_INCL	ZGATEWAY_DELTA_SERVICES	ZGATEWAY_ACCOUNT	NW Gateway POC - CRM Account	☒
/SYCLO/CRMMD_EFI_PARTNER_INCL	ZGATEWAY_DELTA_SERVICES	ZGATEWAY_CONTACT	NW Gateway POC - CRM Contact	☒
/SYCLO/CRMMD_EFI_PARTNER_INCL	ZGATEWAY_DELTA_SERVICES	ZJIRONG_DEMO_EXCHANGE	ZJIRONG DEMO EXCHANGE OBJ	☒

Assignment Detail

Mobile Application:

Exchange Object:

Exch. Object Desc.:

Active Flag: ☒

5 GIS Integration

A geographic information system (GIS) integrates hardware, software, and data for capturing, managing, analyzing, and displaying all forms of geographically referenced information.

The SAP Work Manager application has custom map controls with GIS functionality implemented using SAP Mobile Platform Agentry SDK and Open UI extensions. Their functionality is configured through the Agentry application definitions. The mobile application is delivered with predefined mobile parameters that are Esri-specific, however, you can point to any GIS vendor you choose.

You define all basic configuration for Esri integration through the ConfigPanel.

Enabling and Disabling GIS Parameters from the MDO Configuration

You can enable and disable the GIS parameters through the MDO Configuration page of the Agentry SAP Framework Configuration tool (ConfigPanel).

1. Open the ConfigPanel through the transaction code `/n/syco/configpanel`.
2. Select *Mobile Data Object Configuration* from the Home page.
3. Under *CT - Complex Table*, select the **SWM64_GIS_PARAMETER_LIST**
4. Select the *Data Filter* tab and go to the *GIS_PARAMETERS* filter under the *GET* method.
5. Click the *Change* icon.
6. Highlight the rule you want to change under the Rule List:
 - *Parameter Group*: GIS_Common
 - *Param. Name*: PREF_GIS_ENABLED
 - *Parameter Value*: TRUE
 - *Active Flag*: Active / Inactive
7. Change the rule, then click *Enter* to record the change.
8. *Save* your changes.
9. If applicable, enter the *Transport Request* to include the changes.

GIS with SAP Work Manager

Parameter Group	Parameter Name	Parameter Value
GIS_COLLECTION	PREF_SHOW_CURRENT_LOCATION	True
GIS_COLLECTION	PREF_SHOW_LEGEND	True
GIS_COLLECTION	PREF_SHOW_NEARME	True
GIS_COLLECTION	PREF_SHOW_OBJECT_LIST	True

Parameter Group	Parameter Name	Parameter Value
GIS_COLLECTION	PREF_SHOW_ROUTE	True
GIS_COLLECTION	PREF_SHOW_SEARCH	True
GIS_COMMON	PREF_ARCGIS_RUNTIMECLIENTID	enter client ID
GIS_COMMON	PREF_BASEMAP_NAME_1	Streets
GIS_COMMON	PREF_BASEMAP_NAME_2	Topo
GIS_COMMON	PREF_BASEMAP_NAME_3	Satellite
GIS_COMMON	PREF_BASEMAP_NAME_4	Gray
GIS_COMMON	PREF_BASEMAP_URL_1	PREF_BASEMAP_URL_STREET
GIS_COMMON	PREF_BASEMAP_URL_2	PREF_BASEMAP_URL_TOPO
GIS_COMMON	PREF_BASEMAP_URL_3	PREF_BASEMAP_URL_SATELLITE
GIS_COMMON	PREF_BASEMAP_URL_4	PREF_BASEMAP_URL_GREY
GIS_COMMON	PREF_BASEMAP_URL_GREY	http:// services.arcgisonline.com/ ArcGIS/rest/services/Canvas/ World_Light_Gray_Base/ MapServer
GIS_COMMON	PREF_BASEMAP_URL_SATELLITE	http:// services.arcgisonline.com/ ArcGIS/rest/services/ World_Imagery/MapServer
GIS_COMMON	PREF_BASEMAP_URL_STREET	http:// sampleserver6.arcgisonline.com /ArcGIS/rest/services/ World_Street_Map/MapServer
GIS_COMMON	PREF_BASEMAP_URL_TOPO	http:// services.arcgisonline.com/ ArcGIS/rest/services/ World_Topo_Map/MapServer
GIS_COMMON	PREF_CLUSTER_RADIUS	20
GIS_COMMON	PREF_EDIT_ALLOWED_GEOMETRY_EQUIPMENT	POLYLINE
GIS_COMMON	PREF_EDIT_ALLOWED_GEOMETRY_FUNCTIONALLOCATION	POLYGON
GIS_COMMON	PREF_EDIT_ALLOWED_GEOMETRY_NOTIFICATION	POINT POLYLINE POLYGON
GIS_COMMON	PREF_EDIT_ALLOWED_GEOMETRY_WORKORDER	POINT POLYLINE POLYGON
GIS_COMMON	PREF_EDIT_ENABLED	True
GIS_COMMON	PREF_FEATURELAYER_DEFAULT_STATE_1	False
GIS_COMMON	PREF_FEATURELAYER_DEFAULT_STATE_2	False
GIS_COMMON	PREF_FEATURELAYER_DEFAULT_STATE_3	False

Parameter Group	Parameter Name	Parameter Value
GIS_COMMON	PREF_FEATURELAYER_NAME_1	Story Points
GIS_COMMON	PREF_FEATURELAYER_NAME_2	EAMFS-Buildings
GIS_COMMON	PREF_FEATURELAYER_NAME_3	EAMFS-Roads
GIS_COMMON	PREF_FEATURELAYER_URL_1	http://services.arcgis.com/ P3ePLMys2RVChkJx/ ArcGIS/rest/services/Story/ FeatureServer/0
GIS_COMMON	PREF_FEATURELAYER_URL_2	http://services.arcgis.com/ P3ePLMys2RVChkJx/ ArcGIS/rest/services/Story/ FeatureServer/1
GIS_COMMON	PREF_FEATURELAYER_URL_3	http://services.arcgis.com/ P3ePLMys2RVChkJx/ ArcGIS/rest/services/Story/ FeatureServer/2
GIS_COMMON	PREF_GIS_OFFLINE_ENABLED	True
GIS_COMMON	PREF_MAP_TYPE	ESRI
GIS_COMMON	PREF_NEARME_RADIUS	10
GIS_COMMON	PREF_NEARME_RADIUS_UNITS	MILES
GIS_COMMON	PREF_ROUTE_URL	http:// sampleserver3.arcgisonline.com /ArcGIS/rest/services/ Network/USA/NAServer/Route
GIS_WORKORDER	PREF_SHOW_CURRENT_LOCATION	True
GIS_WORKORDER	PREF_SHOW_LEGEND	True
GIS_WORKORDER	PREF_SHOW_NEARME	False
GIS_WORKORDER	PREF_SHOW_OBJECT_LIST	False
GIS_WORKORDER	PREF_SHOW_ROUTE	True
GIS_WORKORDER	PREF_SHOW_SEARCH	False

The following BAPIs and MDO are specific to using GIS with SAP Work Manager.

BAPIs:

- **Fetch:** /SMFND/GIS_OBJECT_GEODATA_GET
- **Add:** /SMFND/GIS_OBJECT_GEODATA_CRT
- **Edit:** /SMFND/GIS_OBJECT_GEODATA_UPD

MDO: SWM64_GIS_OBJECT_GEOMETRY_GENERIC

6 Setting Up Data Staging

During the initial sync time of any Agency application, the bulk of the time is spent downloading the complex table required for the application. Depending on the data volume of the table, this sync time can be significantly high. By enabling data staging, the synchronization time for a very large data set can be reduced.

Business case: assume you are using an Agency application for SAP backend and during your go-live or user acceptance phase, you are expecting hundreds of users to concurrently connect to the SMP server and sync data for the first time. In normal scenarios, without data staging, you could see long wait times for users trying to download complex table data. Additionally, there is a lot of load on the SAP backend system. With data staging enabled, sync time can be reduced significantly by pre-populating complex table data ahead of time. The Agency mobile framework supports data staging from the [SMFND 610_700 Add-On Support Package 04](#) and later.

Data staging is only recommended for the initial sync, provided data volumes are justified. Although there is no technical limitation of using it during later stages, data staging poses a different set of challenges to keep data staging tables up to date. You have the option to deactivate data staging after initial go-live or you can continue to use it.

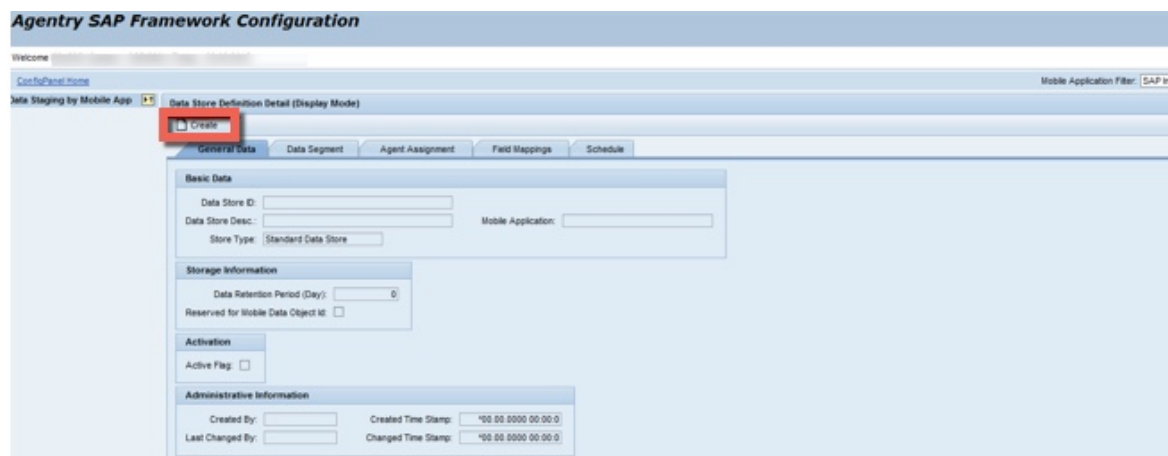
ⓘ Note

Data staging is only supported for complex tables in standard product. By default, data staging is turned off. The use of data staging is only recommended for initial sync, dealing with static data to reduce time during initial sync and any errors due to data staging being out of sync. It is recommended that you take advantage of the xChange framework for subsequent master and transactional data delta syncs.

6.1 General Steps to Activate Data Staging

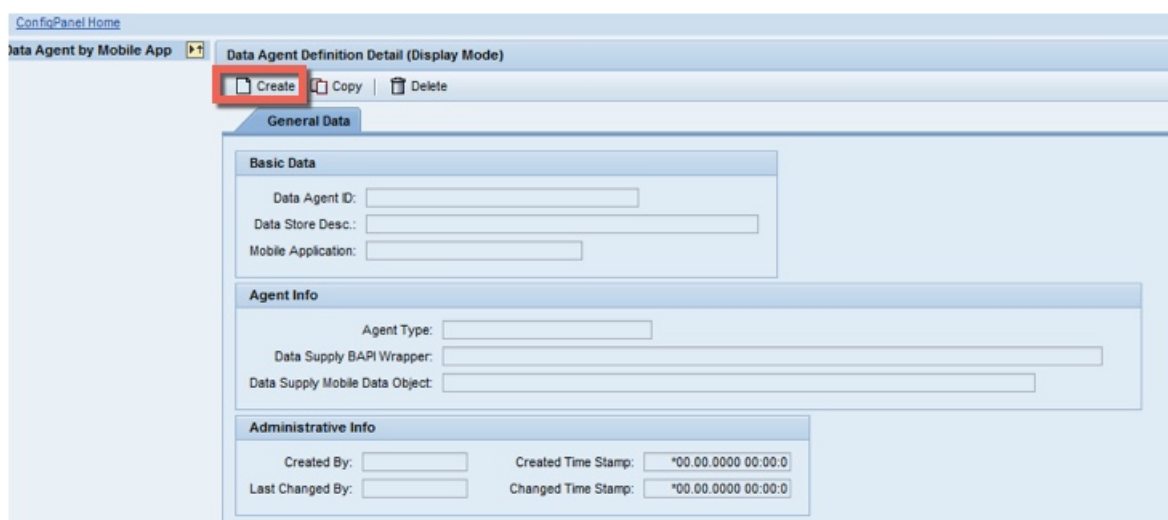
For specific tab or field details, see the appropriate Configuration Panel topic in the application-specific Configuration Guide.

1. From the ConfigPanel Home screen, go to ► [Data Staging Framework Settings](#) ► [Data Store Definition](#) ► and create a new data store definition.



Use the data store definition settings to define the data store settings, data storage, data agent assignment, schedules, and mapping information. A data store is a data storage that can provide data to a mobile data object. A data store can consist of multiple data segments, which are related data sets. A data store is populated by assigned data agents and schedules.

- From the ConfigPanel Home screen, go to [Data Agent Definition](#) and create the data store agent definition.



Use the data store agent settings to define the agent type, data object including the BAPI wrapper, and MDO to be used by the data loading program, which is read during runtime. Then, assign the data agent to a data store.

- From the ConfigPanel Home screen, go to [Mobile Data Object Configuration](#) and create a separate MDO for the data store. The MDO can be the loading MDO or the consuming MDO.

Data Object Detail (Display Mode)

Create Copy Delete Change

General Setting | ResultSet Field Selection | Data Filter | Data Staging | Proxy Setting

Basic Data

Mobile Data Object Id: ZSIM41_DSTORE_MATERIAL_PLANT_VIEW
 Description: ZSIM 4.1.0 - Data Store Load Material master plant view.
 Data Object Type: Complex Table Mobile Application: SAP Inventory Manager 4.1.0
 Reference Business Object:

Data Object Handler Settings

Data Object Handler: /SMERP/CL_MM_MATERIAL_DO : DO Handler: Mate Skip Exception Processing: ☐
 Get Method: GET_MATPLANT
 Create Method:
 Update Method:
 Delete Method:

Exchange Object Settings

Exchange Object: SIM41_MATERIAL Delta Calculation Time Offset (Seconds): 0

Conversion Exit Setting

Enable Conv. Exit Overwrite: ☐

Middleware Reference Info

Reference Middleware Object Type:

Activation

Data Object Active: ☒

You can either create a new MDO from scratch or you can copy an existing standard MDO object, changing the namespace and definition.

- From the ConfigPanel Home screen, go to [BAPI Wrapper Configuration](#) and add the assignment for the data store MDO object at the BAPI wrapper level.

Agency SAP Framework Configuration

Welcome:

ConfigPanel Home

BAPI Wrapper List

- BAPI Wrapper Function Groups
 - /SMERP/BAPI_MM_DELIVERY
 - /SMERP/BAPI_MM_GOODSMOVE
 - /SMERP/BAPI_MM_MATERIAL
 - /SMERPMM_CTMATPLANT_GET**
 - /SMERPMM_CTMATLOC_GET
 - /SMERPMM_CTMATUOM_GET
 - /SMERP/BAPI_MM_MOVEREAS
 - /SMERP/BAPI_MM_MOVETYPE
 - /SMERP/BAPI_MM_PHYSINVDIC
 - /SMERP/BAPI_MM_PLANT
 - /SMERP/BAPI_MM_PROORD
 - /SMERP/BAPI_MM_PURCHORD
 - /SMERP/BAPI_MM_RESERVATION
 - /SMERP/BAPI_MM_SLOC
 - /SMERP/BAPI_MM_VALUEAREA
 - /SMERP/BAPI_MM_VENDOR
 - /SYCLOBAPL_CORE

BAPI Wrapper Detail (Display Mode)

Create Copy Delete Change

General | **Assignment**

BAPI Wrapper Info

BAPI Wrapper Name: /SMERPMM_CTMATPLANT_GET
 Description: BAPI Wrapper: Fetch complex table ctmatPlant

Mobile Data Object Assignment List

Mobile Application	Mobile Data Object Id	Description	Method Type	Active Flag	Default Assignment
SAP_INVENTORY_MANAGER_40	SM40_MATERIAL_PLANT_VIEW	SM 4.0.0 - Material master plant view	GET	☒	☒
SAP_INVENTORY_MANAGER_40	ZSM40_MATERIAL_PLANT_VIEW_CPY	ZSM 4.0.0 - Material master plant view	GET	☒	☐

Assignment Detail

Mobile Application: SAP Inventory Manager 4.0.0
 Mobile Data Object Id: SM40_MATERIAL_PLANT_VIEW-CT - SM 4.0.0 - M
 Description: SM 4.0.0 - Material master plant view
 Method Type: Get method
 Active Flag: ☒

Administrative Info

Created By: Creation Time Stamp:
 Last Changed By: Changed Time Stamp:

You can use the same BAPI for populating staging data and downloading data to the mobile client.

5. From the ConfigPanel Home screen, go to [Mobile Data Object Configuration](#), and on the [Data Staging](#) tab, activate data staging at the consuming MDO object level.

Data Object Detail (Display Mode)

Create Copy Delete Change

General Setting ResultSet Field Selection Data Filter **Data Staging** Proxy Setting

Handler Info

Mobile Data Object Id: SIM41_MATERIAL_PLANT_VIEW Description: SIM 4.1.0 - Material master plant view

Data Object Handler: /SMERPICL_MM_MATERIAL_DO

Data Staging Setting

Common Setting

Disable Conversion Exit for Package Read: ☐

Require Meta Data: ☐

Get Method Setting

Data Staging Supported: ☒

Activate Staging: ☒

Enable Data Store: ☒ Data Store Id: ZSIM41_STORE_MATERIAL_PLANTVIEW :ZSIM 4.1 - Data store for material plant view complex table

Pre-Staging Id:

Package Size: 4.000 Package Validity (Hr): 8

Create Method Setting

Data Staging Supported: ☐

Update Method Setting

Data Staging Supported: ☐

Delete Method Setting

Data Staging Supported: ☐

6. Execute the program SMFND/DSF_DRIVER_PROG to populate the data staging table.
7. To test data staging on the SAP side, execute the BAPI wrapper associated with the complex table.

6.2 Data Staging Activation: Detailed Example

This example shows you the steps necessary to enable data staging for the [ctparts](#) complex table in the SAP Inventory Manager application. However, you can use the steps provided to enable data staging for all applicable mobile applications.

Note

Before you can activate data staging, be sure to install the SAP Mobile add-on with the appropriate B/C set activated. You must also have the correct authorizations to make Configuration Panel changes.

Tip

Experience with ABAP programming is recommended.

Create a new data store definition

From the ConfigPanel Home screen, go to ► [Data Store Definition](#) ► [Data Staging Framework Settings](#) ► and create a new data store definition.

In the data store section of the IMG, you specify all settings related to the data store object, data storage, data agent assignment, mapping information, and scheduling. All of the information you set on these screens are read by the data loading program during runtime. There are five tabs on the [Data Store Definition Detail](#) screen.

- [General Data](#)
- [Data Segment](#)
- [Agent Assignment](#)
- [Field Mappings](#)
- [Schedule](#)

A sample of each tab is shown in this topic, with only key fields described following each screen sample.

Data Store Definition Detail (Create Mode)

General Data | Data Segment | Agent Assignment | Field Mappings | Schedule

Basic Data

Data Store ID: * ZSIM40_MATERIAL_PLANT_VIEW

Data Store Desc.: * IM 4. - Data store for material plant view

Mobile Application: * SAP Inventory Manager 4.

Store Type: Standard Data Store

Storage Information

Data Retention Period (Day): 0

Reserved for Mobile Data Object Id: ☒ SIM40_MATERIAL_PLANT_VIEW - SIM 4. - Material master plant view

Activation

Active Flag: ☒

Administrative Information

Define the basic data and storage information on the [General Data](#) tab. Key fields to note:

[Data Retention Period \(Day\)](#): specify the number of days that you want to retain the data in the staging table.

[Reserved for Mobile Data Object Id](#): This is an optional field. Select the checkbox and specify the consuming MDO object for which you want to restrict this data store object.

[Active Flag](#): be sure to select the checkbox to set the activation flag for the data store.

Data Store Definition Detail (Create Mode)

General Data **Data Segment** Agent Assignment Field Mappings Schedule

Data Store Information

Data Store ID: ZSIM40_MATERIAL_PLANT_VIEW Mobile Application: SAP_INVENTORY_MANAGER_4

Data Supplier Detail

Data Segment ID: * ET_COMPLEX_TABLE

Table Name: /SMERP/STG_MM001 Data staging - Material master plant view complex table

Segment Alias: ET_COMPLEX_TABLE

Administrative Information

Created By: Created Time Stamp: *00.00.0000 00:00:0

Last Changed By: Changed Time Stamp: *00.00.0000 00:00:0

Data Suppliers

+ Add Data Segment - Delete Data Segment

Data Segment	DSegment Table Name	Segment Alias
ET_COMPLEX_TABLE	/SMERP/STG_MM001	ET_COMPLEX_TABLE

The *Data Segment* tab is intended for use with multiple data segments. For instance, if you have a composite complex table with multiple output structures or if the same output table could be populated for different structures.

Data Segment ID: Enter the output BAPI structure for the complex table.

Table Name: Specify staging table name for your scenario. In our example, we are using */SMERP/STG_MM001* table, which is relevant for the ctparts complex table.

Segment Alias: You can specify a segment alias here that is different from the data segment.

To add new data segments, click the *Add Data Segment* button.

Data Store Definition Detail (Change Mode)

General Data | Data Segment | **Agent Assignment** | Field Mappings | Schedule

Data Store Information

Data Store ID: ZSIM40_MATERIAL_PLANT_VIEW Mobile Application: SAP_INVENTORY_MANAGER_4

Mapping Table Detail

Data Agent ID: ZSIM40_AGENT_MATERIAL_PLANT_VIEW - SIM4 - Data Agent for Material Plant View Complex Table
 Data Segment ID: ET_COMPLEX_TABLE Agent Parameter Name: ET_COMPLEX_TABLE - /SYCLOMM_MARAMARC_STR

Administrative Info

Created By: Created Time Stamp: *00.00.0000 00:00:0
 Last Changed By: Changed Time Stamp: *00.00.0000 00:00:0

Table Mappings

Add Table Mapping Delete Table Mapping

Assignment No.	Data Agent ID	Parameter Name	Data Segment ID
001	ZSIM40_AGENT_MATERIAL_PLANT_VIEW	ET_COMPLEX_TABLE	ET_COMPLEX_TABLE

Define your data agent on the [Agent Assignment](#) tab. The number of agents you define depends on the number of data segments you defined previously.

Data Agent ID: Specify your data agent.

Data Segment ID: Specify the data segment you defined in the [Data Segment](#) tab.

Agent Parameter Name: Enter the output structure specified in the Data Agent.

Data Store Definition Detail (Display Mode)

Create Copy Delete Change

General Data | Data Segment | Agent Assignment | **Field Mappings** | Schedule

Data Store Information

Data Store ID: ZSIM40_MATERIAL_PLANT_VIEW Mobile Application: SAP_INVENTORY_MANAGER_4

Field Mapping Detail

Table Mapping Assignment: Data Agent ID: Parameter Name:
 Source Field Name: Target Field Name:

Administrative Information

Created By: Created Time Stamp: *00.00.0000 00:00:0
 Last Changed By: Changed Time Stamp: *00.00.0000 00:00:0

Field Mappings

Table Mapping Assignment No	Data Agent ID	Target Field	Parameter Name	Source Field

Use the fields in the [Field Mappings](#) tab to define the details for your field mapping.

For example, using the functional location complex table from Work Manager, if the field name in the staging table is different from the field name in the target structure (data staging table) and source structure (*ET_COMPLEX_TABLE*), you use the *Field Mappings* tab to map your source and target fields, as shown in the following example:

Field Mappings				
Table Mapping Assignment No	Data Agent ID	Target Field	Parameter Name	Source Field
001	SWM62_AGENT_FUNCLOCATION	SYST_STATUS_CODE	ET_COMPLEX_TABLE	SYSTEM_STATUS_CODE
001	SWM62_AGENT_FUNCLOCATION	TPLMA_EXT	ET_COMPLEX_TABLE	TPLMA_%EXT
001	SWM62_AGENT_FUNCLOCATION	TPLNR_EXT	ET_COMPLEX_TABLE	TPLNR_%EXT

Note

Use this tab for mapping if your source and target fields are different.

Data Store Definition Detail (Display Mode)

Create Copy Delete Change

General Data Data Segment Agent Assignment Field Mappings **Schedule**

Data Store Information

Data Store ID: ZSIM40_MATERIAL_PLANT_VIEW Mobile Application: SAP_INVENTORY_MANAGER_4

Scheduling Detail

Schedule ID: ZSIM40_STORE_MATERIAL_PLANTVIEW

Data Agent ID: ZSIM40_AGENT_MATERIAL_PLANT_VIEW - SIM40 - Data Agent for Material Plant View Complex Table

Start Date: 21.10.2014 End Date: 07.01.2015

Frequency: 1

ECATT Variant Container: Active: ☒

Start Time: 12:00:00 End Time: 14:00:00

Total Executions: 20 ECATT Variant Name:

Administrative Info

Created By: Created Time Stamp: 21.10.2014 17:13:41

Last Changed By: Changed Time Stamp: 21.10.2014 17:13:41

Schedules

Schedule ID	Data Store ID	Data Agent ID	Schedule Begin Time	Schedule End Time	Freq.	Maximum	ECATT Container	ECATT Variant	Active
ZSIM40_STORE_MATERIAL_PLANTVIEW	ZSIM40_MATERIAL_PLANT_VIEW	ZSIM40_AGENT_MATERIAL_PLANT_VIEW	21.10.2014 12:00:00	07.01.2015 14:00:00	1	20			☒

Use the *Schedule* tab to define your scheduling details, including the schedule ID, data agent ID, and the start and end times. You can define a frequency for this schedule as well as an eCATT Variant Name and eCATT Variant Container. These scheduling parameters are read by the loading utility program /SMFND/DSF_DRIVER_PROG for the data store at the runtime

Define the data store agent

From the ConfigPanel Home screen, go to *Data Agent Definition* and create the data store agent definition.

In addition to the agent type, specify data objects like the BAPI wrapper and MDO object to be used by the data loading program that is read at runtime.

Data Agent Definition Detail (Change Mode)

General Data

Basic Data

Data Agent ID: * ZSIM41_AGENT_MATERIAL_PLANT_VIEW

Data Store Desc.: * SIM41 - Data Agent for Material Plant View Complex Table

Mobile Application: * SAP Inventory Manager 4.1.0

Agent Info

Agent Type: BAPI Wrapper Data Supplier

Data Supply BAPI Wrapper: /SMERPMM_CTMatPlant_GET : BAPI Wrapper: Fetch complex table ctMatPlant

Data Supply Mobile Data Object: SIM41_MATERIAL_PLANT_VIEW : GET : SIM 4.1.0 - Material master plant view

Administrative Info

Here, you define the data store supplying agent settings. Within this IMG activity, you define a data staging framework data agent. A data agent defines the technical routine that populates a data store. The technical routine definitions include class handler and function module.

Data Agent ID: Enter the data agent object ID.

Data Store Desc.: Enter a description for the data store.

Mobile Application: Specify the mobile application for which the data agent is applicable.

Agent Type: Select the agent type. In this example we are using the BAPI for the complex table.

Data Supply BAPI Wrapper: Select the BAPI wrapper to be used for the loading staging table.

Data Supply Mobile Data Object: Select the data store MDO that you created in the previous step.

Create a separate MDO object for the data store

From the ConfigPanel Home screen, go to [Mobile Data Object Configuration](#) and create a separate MDO for the data store.

→ Tip

You can either create a new MDO from scratch or you can copy an existing standard MDO object, changing the namespace and definition.

Data Object Detail (Display Mode)

Create Copy Delete Change

General Setting ResultSet Field Selection Data Filter Data Staging Proxy Setting

Basic Data

Mobile Data Object Id: ZSIM41_DSTORE_MATERIAL_PLANT_VIEW
 Description: ZSIM 4.1.0 - Data Store Load Material master plant view
 Data Object Type: Complex Table Mobile Application: SAP Inventory Manager 4.1.0
 Reference Business Object:

Data Object Handler Settings

Data Object Handler: /SMERP/CL_MM_MATERIAL_DO : DO Handler: Mate Skip Exception Processing: ☐
 Get Method: GET_MATPLANT
 Create Method:
 Update Method:
 Delete Method:

Exchange Object Settings

Exchange Object: SIM41_MATERIAL Delta Calculation Time Offset (Seconds): 0

Conversion Exit Setting

Enable Conv. Exit Overwrite: ☐

Middleware Reference Info

Reference Middleware Object Type:

Activation

Data Object Active: ☒

Administrative Info

You can define a separate set of data filters for the data store MDO object. In this example, we have hard-coded plants values for the material data to be loaded. When the loading program is executed, it looks at these filters and loads material data only for these plants values into the staging table.

Data Object Detail (Display Mode)

Create Copy Delete Change

General Setting Result Set Field Selection **Data Filter** Data Staging Proxy Setting

Handler Info

Mobile Data Object Id: ZSIM41_DSTORE_MATERIAL_PLANT_VIEW Description: ZSIM 4.1.0 - Data Store Load Material master plant view.
Data Object Handler: /SMERP/CL_MM_MATERIAL_DO

Data Object Filter Rule Definition

Defined Filters

Get Method - GET_MATPLANT

Rule Editor

Method Name: GET_MATPLANT Filter Name: PLANT
Reference Table Name: MARC Reference Field Name: WERKS
Data Filter Rule Key: ZSIM41_DSTORE_MATERIAL_PLANT_VIEW.GET.PLANT
DOF Rule Type: Static Value In Range Format

Enter Range Value

Sign: Inclusive Option: =
Low Value: 1100 - Berlin
High Value:
Active Flag: ☒

Rule List

Rule No.	Rule Type	Rule Value	Active Flag
00001	RANGE	IEQ1100	☒
00002	RANGE	IEQ2000	☒
00003	RANGE	IEQ3000	☒
00004	RANGE	IEQ	☒

There is no need to enable any parameters in the *Data Staging* tab for the data store MDO.

Add assignment for the data store MDO object to the BAPI wrapper

From the ConfigPanel Home screen, go to *BAPI Wrapper Configuration* and add the assignment for the data store MDO object at the BAPI wrapper level.

ConfigPanel Home Mobile Application Filter

BAPI Wrapper List

- BAPI Wrapper Function Groups
 - /SMERP/BAPI_MM_DELIVERY
 - /SMERP/BAPI_MM_GOODSMOVE
 - /SMERP/BAPI_MM_MATERIAL
 - /SMERP/BAPI_MM_MATPLANT_GET**
 - /SMERP/BAPI_MM_MATPLANT_GET
 - /SMERP/BAPI_MM_MOVEAS
 - /SMERP/BAPI_MM_MOVETYPE
 - /SMERP/BAPI_MM_PHYSNDOC
 - /SMERP/BAPI_MM_PLANT
 - /SMERP/BAPI_MM_PROCOORD
 - /SMERP/BAPI_MM_PURCHORD
 - /SMERP/BAPI_MM_RESERVATION
 - /SMERP/BAPI_MM_SLOC
 - /SMERP/BAPI_MM_VALUEAREA
 - /SMERP/BAPI_MM_VENDOR
 - /SYCLO/BAPI_CORE
 - /SYCLO/BAPI_MM

BAPI Wrapper Detail (Display Mode)

Create Copy Delete Change

General **Assignment**

BAPI Wrapper Info

BAPI Wrapper Name: /SMERP/BAPI_MM_MATPLANT_GET
Description: BAPI Wrapper: Fetch complex table cMatPlant

Mobile Data Object Assignment List

Mobile Application	Mobile Data Object Id	Description	Method Type	Active Flag	Default Assignment
SAP_INVENTORY_MANAGER_41	SM41_MATERIAL_PLANT_VIEW	SM 4.1.0 - Material master plant view	GET	☒	☒
SAP_INVENTORY_MANAGER_41	ZSIM41_DSTORE_MATERIAL_PLANT_VIEW	ZSIM 4.1.0 - Data Store Load Material master plant view.	GET	☒	☐

Assignment Detail

Mobile Application: SAP Inventory Manager 4.1.0
Mobile Data Object Id: SM41_MATERIAL_PLANT_VIEW-CT - SM 4.1.0 - M
Description: SM 4.1.0 - Material master plant view
Method Type: Get method
Active Flag: ☒

→ Tip

It is most common to use an existing BAPI wrapper to load the data.

You can use the same BAPI for populating staging data and downloading data to the mobile client. Make sure that the loading or consuming MDO is set as the default assignment.

Activate data staging at the consuming MDO object level

From the ConfigPanel Home screen, go to [Mobile Data Object Configuration](#) and on the [Data Staging](#) tab, activate data staging at the consuming MDO object level.

Data Object Detail (Display Mode)

Create | Copy | Delete | Change

General Setting | ResultSet Field Selection | Data Filter | **Data Staging** | Proxy Setting

Handler Info

Mobile Data Object Id: SIM41_MATERIAL_PLANT_VIEW | Description: SIM 4.1.0 - Material master plant view
Data Object Handler: /SMERPICL_MM_MATERIAL_DO

Data Staging Setting

Common Setting

Disable Conversion Exit for Package Read: ☐
Require Meta Data: ☐

Get Method Setting

Data Staging Supported: ☒
Activate Staging: ☒
Enable Data Store: ☒ | Data Store Id: ZSIM41_STORE_MATERIAL_PLANTVIEW :ZSIM 4.1 - Data store for material plant view complex table
Pre-Staging Id:
Package Size: 4,000 | Package Validity (Hr): 8

Create Method Setting

Data Staging Supported: ☐

Update Method Setting

Data Staging Supported: ☐

Delete Method Setting

Data Staging Supported: ☐

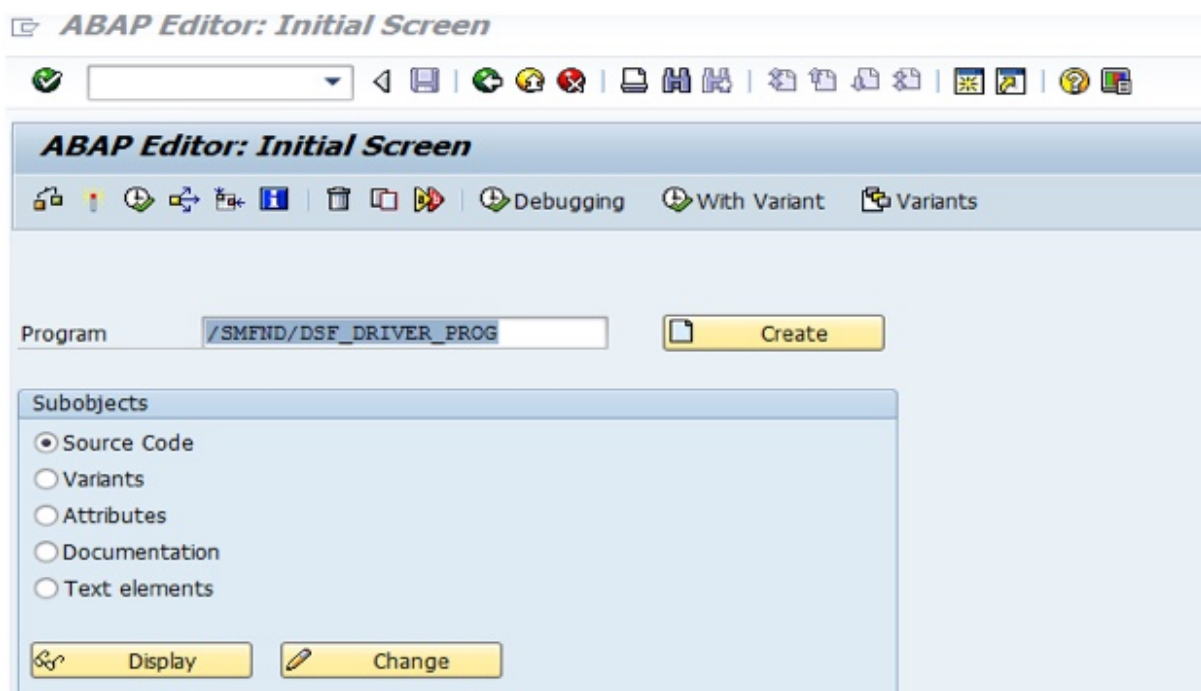
Execute the loading program to populate the data staging table

To execute the loading program for the data store, schedule the data store loading utility program SMFND/DSF_DRIVER_PROG as an optional background job or standalone. This program can be used for both loading and purging the staging table data.

ⓘ Note

Before you execute the loading program, make sure that the required configurations are in place.

First, execute the program SMFND/DSF_DRIVER_PROG in [SE38](#).



Next, specify inputs for the loading program. Remember that this single program can be used to perform both data loading and data purging.

Note

Be sure to apply <https://me.sap.com/notes/2152313> before using the qRFC option. Data loading or purging using qRFC does not work consistently in the data loading program `/SMFND/DSF_DRIVER_PROG`.

Data Staging Framework - Data Loading Agent

☒ Perform Data Loading

Mobile Application: SAP_INVENTORY_M... to [] [↔]

Data Store ID: ZSIM4..._MATERIAL... to [] [↔]

Data Agent ID: ZSIM4..._AGENT_MA... to [] [↔]

Schedule ID: TERIAL_PLANTVIEW... to [] [↔]

☐ Ignore Validity (Scheduling)

☐ Load Using qRFC Tasks

☐ Perform Data Purging

Mobile Application: [] to [] [↔]

Data Store ID: [] to [] [↔]

☐ Ignore Validity (Retention)

☐ Purge Using qRFC Tasks

☒ Display Errors Only

The data staging loading program can be executed wide open for a given mobile application data store ID or data agent ID.

Perform Data loading: Check this option if you want the program to execute for data loading.

Mobile Application: Specify the name of the mobile application.

Data store ID, Data Agent ID, and Scheduling ID are all optional fields.

Ignore Validity (Scheduling): Select this checkbox if you want the loading program to ignore any schedule settings defined under the **Schedule** section of the data store.

Load Using qRFC Tasks: Select this checkbox to load the data staging table for multiple data store data agent IDs. By default, the program executes in sequential fashion by checking the qRFC option. The program triggers parallel processing when executing the program for multiple data store or data agent IDs.

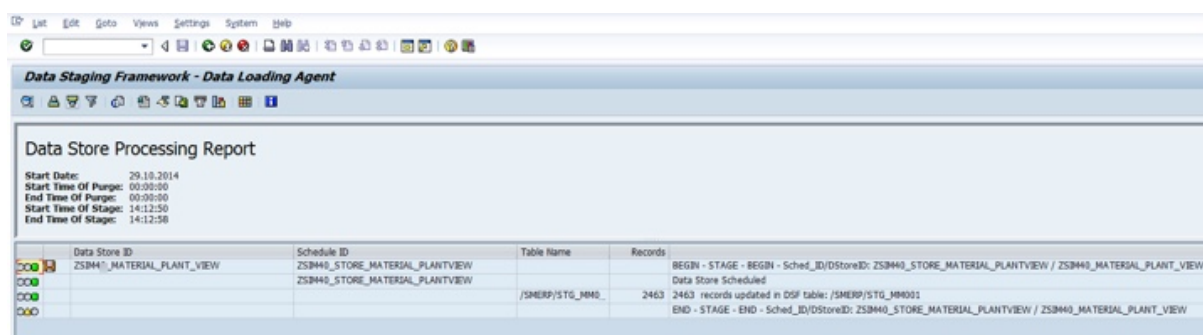
Perform Data Purging: Select this option if you want the program to execute for data purging.

Ignore Validity (Retention): Select this checkbox if you want the loading program to ignore any retention settings defined under the General tab of the data store.

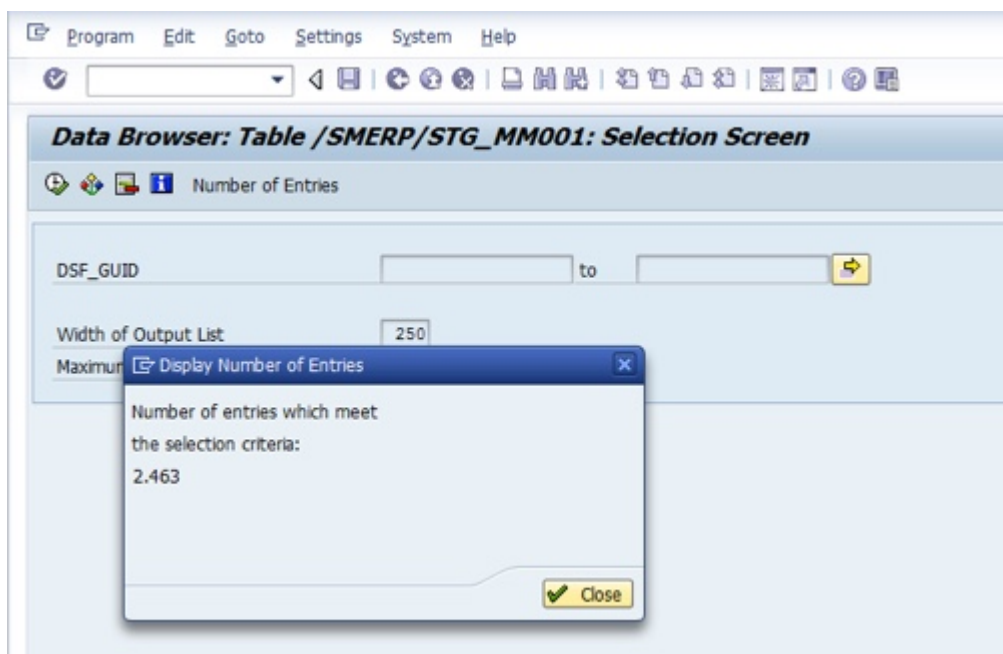
The following example is a look at the material staging table before executing the loading program:



At the end of the program execution, it displays a log outlining the processing status, as shown in the example:



Finally, the number of material staging entries is displayed after the loading program is executed.



Test data staging

To test data staging on the SAP side, execute the BAPI wrapper associated with the complex table.

6.3 Troubleshooting, FAQs, and Resources

Check here for troubleshooting tips, answers to frequently asked questions, and additional resources for data staging.

Troubleshooting Tips

Issue: The data staging table is populated but records are not retrieved by the consuming BAPI for the complex table.

Resolution:

- Make sure that [DSF_RECORD_ID](#) field has a sequential numeric value in the respective complex table.
- If you are not seeing a sequential numeric value for the [DSF_RECORD_ID](#) field, then most likely, the number range for [/SMFND/DS1](#) object is not maintained.

Issue: The number range for object [/SMFND/DS1](#) is maintained, but the records are not retrieved by the consuming BAPI for the complex table.

Resolution: Make sure that the number range interval is defined as [01](#).

Issue: The number range is maintained correctly, but the records are not retrieved by the consuming BAPI for the complex table.

Resolution:

- Check the configuration related to the consuming MDO object to make sure that data staging is activated.
- Check the configuration related to data agent and data store.
- Make sure that the load program was executed and records are populated in the data staging table.
- Make sure that you are running with support package [04](#) with add-on [SMFND 610_700](#).

Issue: Tried loading data using the qRFC option, but no records are loaded into the staging table.

Resolution: Check the qRFC queue using transaction code [SMQ2](#) to check for errors.

Issue: Data is populated in the data staging table but the latest data that was changed is not coming down to the Agentry client.

Resolution: Data staging tables are static in nature and does not have an exchange mechanism. Therefore, the tables only contain a snapshot at the point when the loading program was executed. Any data that is changed after the loading program was executed will not be found in the staging table.

Issue: The loading program was executed by the log indicates that zero records are loaded.

Resolution: Carefully review your loading program options and other related data staging configuration to determine the cause.

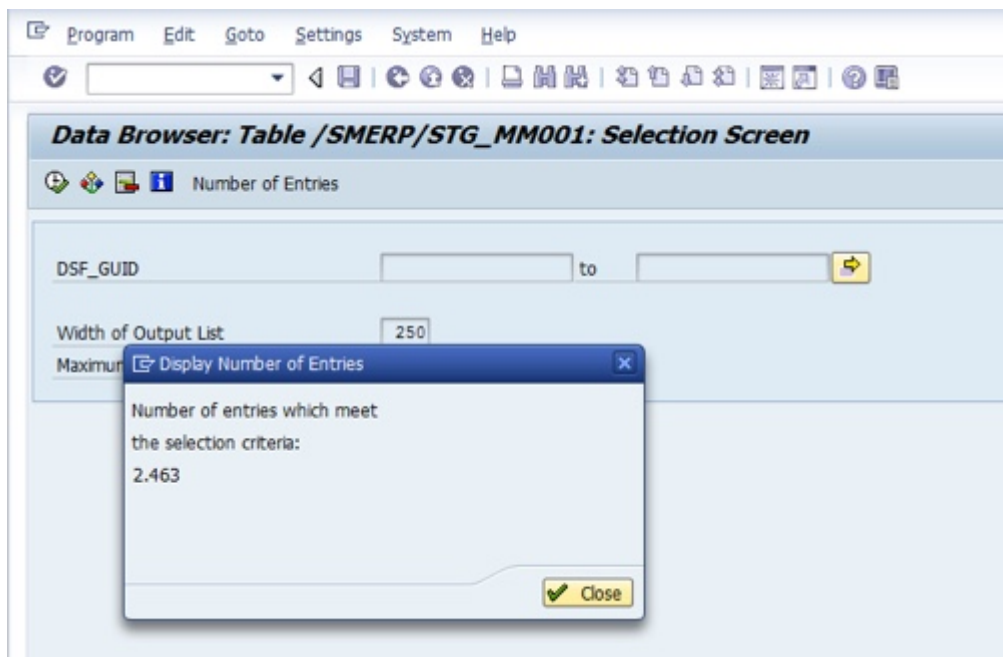
Issue: In spite of enabling data staging, the initial Agentry client sync takes a long time.

Resolution: By enabling data staging, you are only speeding up the complex table download time. Analyze and identify where the application is taking maximum time during the initial sync process.

Frequently Asked Questions

- After the initial sync is completed, does the data staging framework use an exchange mechanism to load delta changes?
Data is read from the staging table only during the initial sync. Then, data is read from exchange table for delta changes.
- Can we disable data staging after the initial go live?
Yes – Once data staging is disabled, the standard complex table logic flow is used during subsequent regular syncs. Any initial sync during this period may take time since data is read by executing a query on the SAP backend and not from the staging table. To disable data staging, go to the consuming MDO object staging tab and disable data staging there. Transport the changes across system landscape.
- Can we use data staging even after go live or after initial syncs of Agentry clients?
We do not recommend that using data staging after initial syncs are completed. However, there are no restrictions from the application side as long as the staging table is updated with the changes.
- Can we implement or enable data staging for Z complex tables?
Yes – The SAP add-on framework provides an option to enable data staging even for custom Z complex tables.
- Can we implement data staging for other Agentry objects, such as transactions?
The standard product provides data staging support only for complex tables. The exchange mechanism is available for transaction objects, so there is no need to enable data staging. But technically there is restriction from application side not to use data staging for transaction objects, which needs careful review before implementing any such logic.
- Can we create our own data staging tables?

Yes, you can create your own staging tables. Make sure that any Z staging tables contain all these standard fields:



- Is data staging specific to any Agency product?
No – It is not specific to any standard product, but the data staging framework was introduced with add-on SMFND 610_700 with support package 04 onwards. Data staging is only enabled for a few complex tables at this point in each of the mobile applications.
- If we have a rollout for Agency applications planned in phases, does activating data staging in the middle effect users who have already have gone live?
Once data staging is enabled, it is enabled at the application level. Be sure to populate the staging table with data required for all users. Any initial sync or client reset from an application user attempts to retrieve data from staging tables. Therefore, you may encounter data quality issues if the staging table is not up to date.
- Can we create our Z loading program to load staging programs?
We do not recommend that you develop your own loading program. However, there is no restriction on application side to copy or create your own Z loading program as long it follows the framework of the standard data staging program. Once you create a Z program, it becomes your responsibility to upgrade and maintain it.
- Can we populate the data staging table from multiple sources?
Yes – Agency data staging provides an option to update a single-target staging table with multiple sources.
- How can I verify that the data loading program populated the staging table?
During program execution, the program log indicates the number of records loaded into staging table and contents of staging table can always be checked using [SE16 / SE11](#).
- Can we load staging tables with data for multiple plants? And if so, when users connect during initial sync, will they get data for their respective plant based on MDO object rules for complex table?
Yes - It works. For MDO object of data store, specify all the required plants for which data needs to be loaded to a complex table. When users connect and try to sync, the data staging framework looks at the consuming MDO object rules and loads the data.
- Is there a way to verify that the data supplying BAPI is retrieving records from the staging table?

Yes – In SE37, you can use the BAPI to retrieve the complex table and do unit testing to make sure that data is pulled from the data staging table. Or, if you are familiar with ABAP debugging, you can debug the class handler code to check if data staging code is invoked.

Additional Resources

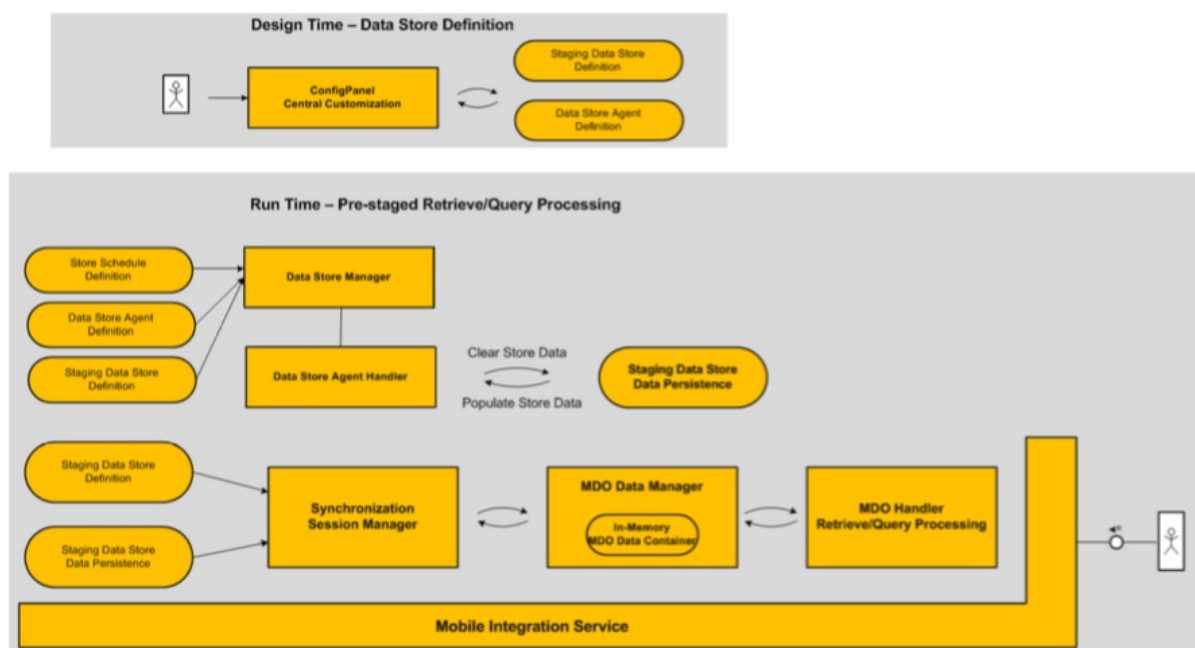
- See SAP Note [2116192](#): FAQ - Mobile add-on for SAP Performance Recommendations.
- Refer to the sizing guidelines for both the SAP Mobile Platform and the applicable mobile product.
- Refer to the Configuration Guide for the applicable mobile product.

6.4 Data Staging Flows and Predefined Tables

This topic contains high-level diagrams depicting the runtime and design flow for data staging and the data staging sequence. It also contains a list of the predefined data staging tables that come delivered with the mobile application solution.

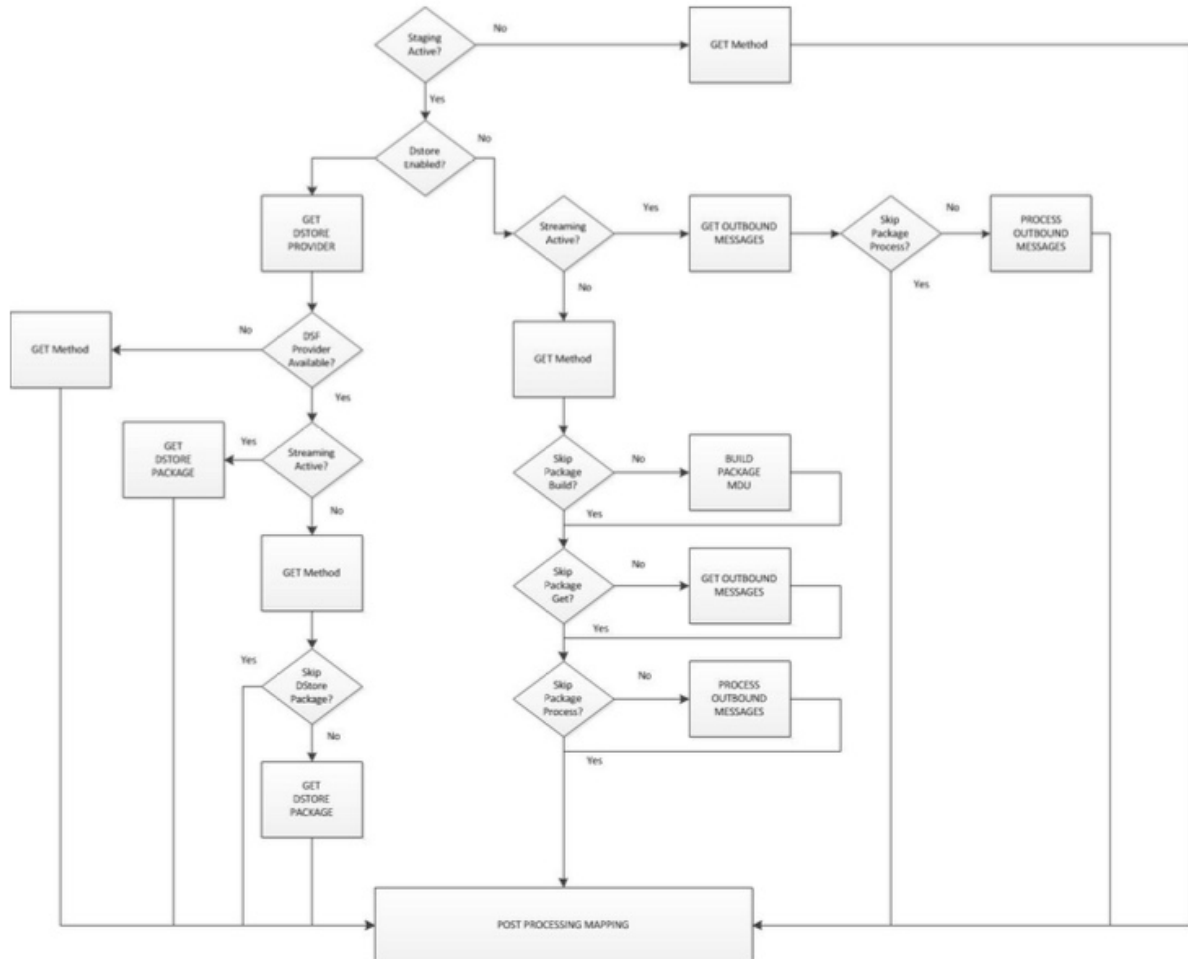
Runtime and Design Flow

At a high-level, the following diagram shows the runtime and design flow for data staging.



Data Staging Sequence

The following diagram depicts the general data staging sequence.



Predefined Staging Tables

The following list shows the predefined data staging tables that are delivered with the mobile solution.

Table Name	Short text
/SMERP/STG_MM001	Data staging - Material master plant view
/SMERP/STG_MM002	Data staging - Material storage location v
/SMERP/STG_MM003	Data staging - Material UoM view complex t
/SMERP/STG_MM004	Data staging - Material warehouse view com
/SMERP/STG_MM005	Data staging - Material batch view complex
/SMERP/STG_PM001	Data staging - Equipment complex table
/SMERP/STG_PM002	Data staging - Functional Location complex
/SMERP/STG_PM003	Data staging - Equipment List Set - Hdr -
/SMERP/STG_PM004	Data staging - Equipment list set - Measur
/SMERP/STG_PM005	Data staging - Equipment LSet - DMS Doc. O
/SMERP/STG_PM006	Data staging - Equip List Set - Classific
/SMERP/STG_PM007	Data staging - Equip List Set - Char Value
/SMERP/STG_PM008	Data staging - Functional Loc. List Set -
/SMERP/STG_PM009	Data staging - Func. Loc. List Set - Meas.
/SMERP/STG_PM010	Data staging - Func. Loc. LSet - DMS Doc.
/SMERP/STG_PM011	Data staging - Func Loc. LSet - Classific
/SMERP/STG_PM012	Data staging - Func Loc LSet - Char Values

7 Inbound Transaction Management

Inbound transaction management is critical to ensure that data updates to the backend SAP system are posted properly and data integrity is preserved.

Inbound transaction management provides the functionality of asynchronous transaction processing. This functionality allows mobile client transactions to be uploaded and checked into inbound transaction queues in the SAP system. Transactions parked in the inbound transaction queues are processed asynchronously at a later time.

Inbound transaction management can be used for regular transaction processing or transaction error handling. Inbound transaction management can be enabled based on object type. A hybrid mode of mixing asynchronous transaction processing with synchronous transaction processing based on object types is also supported.

Data entries from mobile applications are captured as mobile transactions. When connection to backend SAP system is established from mobile device, mobile transactions are uploaded and posted into SAP system. From SAP system perspective, we refer to this process as mobile transaction management. Mobile transaction management is critical to ensure that data updates to backend SAP system are posted correctly and preservation of SAP system data integrity.

Mobile transactions can be processed synchronously or asynchronously when being posted into SAP system. In this chapter of Inbound transaction management, we explain the functionality of asynchronous transaction processing. In asynchronous transaction processing, mobile transactions are uploaded and parked into inbound transaction queues in the SAP system. Transactions stored in the inbound transaction queues are then processed asynchronously at a later time.

Inbound transaction management can be used for regular transaction processing or transaction error handling. A hybrid mode of combining asynchronous transaction processing with synchronous transaction processing is supported based on transaction object types.

Overall inbound transaction management configuration and management consists of five key areas:

- Inbound transaction queue configuration for the mobile application
- BAPI wrapper enablement
- Inbound transaction processing
- Inbound transaction administration and monitoring
- Security

7.1 Inbound Transaction Queue Configuration

The inbound transaction queue is a configuration object that you define using the Configuration Panel (ConfigPanel) customizing tool.

The inbound transaction queue defines the types of transactions that can be checked into the queue and how those transactions are to be managed and processed.

Before an inbound transaction queue can be created, inbound transaction management must be enabled for the mobile application. While inbound transaction queues are mobile application-specific, you can define multiple standard queues for each application. Each standard inbound transaction queue only accepts specific types of transactions, based on the queue assignment settings. When you define multiple inbound transaction queues, you can optionally designate one queue as the default inbound queue for the mobile application. The default inbound queue only accepts types of inbound transactions that have no standard inbound transaction queue assigned to them.

Transaction Queue Assignment

In order for an inbound transaction to be processed asynchronously, the inbound transaction must be assigned to an inbound queue. Queue assignment is based on the technical object/interface that generates the transaction.

Note

BAPI Wrapper is the only supported type of technical object/interface at this time.

For each standard inbound transaction queue, you need to define a list of accepted BAPI Wrappers. A BAPI Wrapper list is not required for the default inbound transaction queue of the mobile application.

Tip

In addition to BAPI Wrapper restrictions, you can also define additional transaction attributes such as transaction object type, object group, and object group 1 as criteria for queue assignment.

Transaction Queue Handler

The transaction queue handler provides the technical implementation that manages the following inbound queue related tasks:

- transaction input data parsing
- transaction check-in process
- transaction check-out/edit process
- transaction cancellation
- transaction approval
- transaction key-mapping
- transaction processing

You must assign a transaction queue handler to the inbound transaction queue.

All inbound queue handlers must be sub-classed from the queue handler base class [/SMFND/CL_IBQ_INBQUEUE_MGR_ABS](#). A general purpose queue handler [/SMFND/CL_INB_Q_MGR_STD](#) is provided to support standard inbound queue tasks. Specialized inbound queue handlers can be developed to handle object type-specific inbound queue requirement, such as key-mapping, approval, processing, and so on.

7.2 BAPI Wrapper Enablement

Inbound transaction queue assignments are based on the technical object/interface that generates the transaction, such as the BAPI Wrapper.

To support inbound transaction management, BAPI Wrappers that are defined based on Mobile Application Integration Framework protocol must also have the following parameters defined:

- Table Parameter [IT_TRANSACTION_REQ_INFO](#), type [/SMFND/IBQ_TRANS_REQ_INFO_TAB](#). This is an input parameter and must be populated by the calling application.
- Table Parameter [ET_TRANSACTION_REQ_STATUS](#), type [/SMFND/IBQ_TRANS_REQ_STA_TAB](#). This is an output parameter. It returns with the inbound transaction status information after a transaction is checked into an inbound transaction queue.

7.3 Inbound Transaction Processing

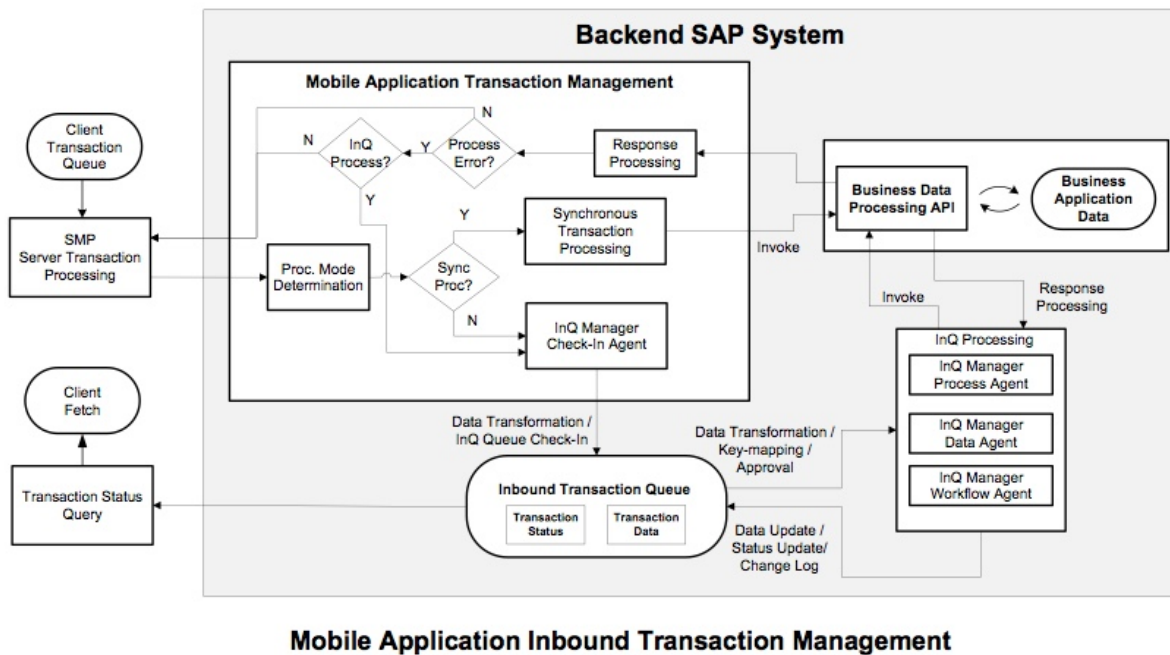
Before you can look at inbound transaction processing, first understand the general lifecycle of inbound transaction management.

Inbound transaction management is critical to ensure that data updates to the backend SAP system are posted properly and data integrity is preserved. In order for any transaction to be checked into an inbound queue, enable inbound transaction management for the mobile application.

A transaction in an inbound queue can go through several unique status phases.

Status	Description
NEW	Transaction is checked into the queue, no approval required.
REVIEW	Transaction is checked into the queue, and approval workflow is required for the checked-in transaction.
ERROR	Transaction is processed and resulted in error.
READY	Transaction input data is modified and ready to be reprocessed.
CANCEL	Transaction is canceled and can no longer be processed. Purge the transaction.
COMPLETE	Transaction is processed successfully, and is purged.

You can assign an expiration time to an inbound transaction. Expired transactions cannot be processed. Purge the expired transactions.



Inbound Transaction Processing

Users running the same mobile application can choose between synchronous transaction processing or asynchronous transaction processing individually. When you enable the *Require User Parameter* option for check-in mode, only users with the matching user *Parameter ID* and *Activation Token* value in their SAP user profile have inbound transaction processing enabled.

The *Check-in Mode* setting of the inbound queue configuration determines how eligible transactions can be checked into a queue. An eligible transaction is a transaction with queue assignment defined.

- *Always Check-in*: Always checks the transaction into the queue for processing.
- *Dependent Transaction Only*: Checks if there are any dependent transactions already in the queue. If so, the transaction is checked in. If not, the transaction skips the queue and is processed synchronously.
- *Error Processing Only*: Checks the transaction into the queue only if there is an error. Otherwise, the transaction skips the queue and is processed.

Processing of an inbound transaction deals with various requirements, such as:

- Dependency transaction conditions
- Key mapping requirements
- Conflict resolution
- Error handling
- Processing history and reprocessing

Dependent Transaction Processing

For each inbound transaction checked into the inbound queue, a dependent transaction can be identified. If a dependent transaction is identified, the inbound transaction can only be processed after the dependent transaction reaches a permissible state (*COMPLETE*, *CANCEL*, or time expired).

Dependent transaction for an inbound transaction is identified based on inbound transaction attributes, *Dependency Scope Setting*, and time sequence of transactions being submitted. If multiple dependency transactions are identified, only the most recent dependency transaction is selected as the dependency transaction. The dependency transaction GUID is recorded in the attribute *DEP_TRANS_GUID* of the inbound transaction when checked into the inbound queue.

The Dependency Scope Setting has the following options:

Scope Option	Description
<i>Cross Application</i>	Search dependent transactions in all of the inbound queues across all mobile applications.
Mobile Application (<i>Application Only</i>)	Search dependent transactions in the inbound queues of the current inbound transaction mobile application.
Mobile User (<i>User Only</i>)	Search dependent transactions in the inbound queues of the current inbound transaction mobile application and submitted by the current mobile user.

There are several inbound transaction attributes used for dependent transaction determination.

Attributes	Usage
<i>DEP_TRANS_GUID</i> or <i>DEP_TRANS_ID</i>	The client application can set the dependent transaction GUID (<i>DEP_TRANS_GUID</i>) when submitting the inbound transaction. Or, the dependent transaction GUID can be set during the check-in process if it is determined by the inbound queue handler.
<i>DEP_OBJECT_TYPE</i> and <i>DEP_OBJECT_KEY</i>	Dependent object type and dependent object key
<i>OBJTYP_REF</i> and <i>OBJKEY_REF</i>	Reference object type and reference object key
<i>OBJECT_TYPE</i> and <i>OBJECT_KEY</i>	Object type and object key, which can uniquely identify the object being submitted in the inbound transaction.

Key-Mapping Requirement in Dependency Transaction Processing

In a complex dependency transaction processing scenario, two dependent local transactions are created on the mobile client in offline mode. For example, consider the following sample scenario:

Transaction A with temporary local ID *LOCAL_1* assigned. Transaction B is a follow-up transaction with temporary local ID *LOCAL_2* assigned. Transaction B has reference to transaction A using its local ID *LOCAL_1*.

For example, an equipment master record is created from mobile application as transaction A with local equipment ID [LOCAL_1](#). A follow-up work order transaction is created as transaction B with local work order ID [LOCAL_2](#). Transaction B makes reference to the newly created equipment using its temporary local equipment ID [LOCAL_1](#).

When the transactions from the mobile application are processed, transaction A is uploaded into the SAP system and processed first and an SAP equipment ID [EQUIPMENT_1](#) is generated after the equipment master record is created in the SAP system. Before transaction B can be successfully processed, replace the reference to the local equipment ID [LOCAL_1](#) with the actual SAP equipment ID [EQUIPMENT_1](#). This step is known as key mapping. Without key mapping, posting of transaction B results in error.

For this scenario, consider the following [Key-Mapping](#) cross-reference information:

During processing of transaction A, capture information about local equipment ID [LOCAL_1](#) and the newly generated SAP equipment ID [EQUIPMENT_1](#) for later use. This information is stored into system table [/SMFND/D_INB_R](#) by the inbound queue handler based on the cross reference customizing settings defined for the inbound transaction queue.

These fields are included in the [Cross Reference](#) table:

Field	Description
TRANSACTION_GUID	Inbound transaction unique identifier
MOBILE_APP	Mobile application of the inbound transaction
OBJECT_TYPE	Object type of the inbound transaction
SRC_OBJECT_KEY	Local object ID of the object being created
TGT_OBJECT_KEY	SAP object ID of the object after created in SAP system

The following are the [Cross Reference](#) settings:

Attribute	Description
Object Type – Source	Source value of the object type. If Constant Value is selected, specify a constant value in Object Type – Path . If Path is selected, use Object Type – Path and Object Type – Attribute to define the BAPI parameter and attribute from which the value should be taken.
Object Type – Path	Contains either a constant value or the name of an input BAPI parameter.
Object Type – Attribute	Contains the name of an input BAPI parameter attribute.

Attribute	Description
<i>From Key – Source</i>	Source value of the source object key. If <i>Constant Value</i> is selected, then specify a constant value in <i>From Key – Path</i>. If <i>Path</i> is selected, then use <i>From Key – Path</i> and <i>From Key – Attribute</i> to define the BAPI parameter and attribute from which the value should be taken.
<i>From Key – Path</i>	Contains either a constant value or the name of an input BAPI parameter.
<i>From Key - Attribute</i>	Contains the name of an input BAPI parameter attribute.
<i>To Key - Source</i>	Source value of the new SAP object ID. If <i>Constant Value</i> is selected, specify a constant value in <i>To Key – Path</i>. If <i>Path</i> is selected, use <i>To Key – Path</i> and <i>To Key – Attribute</i> to define the BAPI parameter and attribute from which the value should be taken.
<i>To Key – Path</i>	Contains either a constant value or the name of the BAPI parameter.
<i>To Key - Attribute</i>	Contains the name of an output BAPI parameter attribute.

Next, consider key mapping value replacement. Inbound transaction attribute values are replaced before being posted into the SAP system by the inbound queue handler based on information captured in the Cross Reference table and the key mapping customizing settings.

Attribute	Description
<i>Mapping Type</i>	How value replacement should be handled. Only <i>Replace</i> option is possible.
<i>Reference – Object Type</i>	Object type of the cross reference record. This info and dependent transaction GUID is used by inbound queue handler to search for the specific cross reference info record.
<i>Target Key Reference Path</i>	Contains the name of an input BAPI parameter attribute.
<i>Target Key – Attribute</i>	Attribute name for which input value should be replaced by cross reference info.

Conflict Resolution

When posting an inbound transaction, conflict situation can arise because the SAP object being updated has been changed in the back-end system since a copy is downloaded to the mobile client.

The *Conflict Mode* setting defines how the conflict situation is to be handled. There are four options:

- *Server Always Win*: When selected, your changes cannot be processed; the server copy is kept intact. If there is any backend change made since a copy of the SAP object is downloaded to the mobile client, retain the backend change and discard the client transaction.
- *Last In Win*: When selected, the last update made is kept and other changes are overridden. This selection is the same as client always win. The SAP object is posted with the mobile client change captured in inbound transaction.
- *Manual*: This option does not allow any transactions to be processed until manually done from the back end. Inbound transactions remain parked in the inbound transaction queue. Manually review the inbound transaction data using the inbound monitor tool and decide whether to post or discard the transaction.
- *Not Relevant*: This option indicates that conflict is not relevant and works the same way as *Last in Win* but has a slighter faster performance. Mobile client changes captured by inbound transaction are posted to the SAP object.

Error Handling, Processing History, and Reprocessing

Transaction processing history with return info is captured automatically by the inbound queue handler and saved for later review. In case error occurs during posting of an inbound transaction, error messages are also captured as part of processing history.

Use the Inbound Transaction Monitor tool to view the transaction processing history. Transactions with errors can be reprocessed. You can edit the transactions before reprocessing as well.

7.4 Inbound Transaction Administration

The Inbound Transaction Monitor is the central tool provided to monitor and manage any inbound transaction in an inbound queue.

The Transaction Inspector tool, which is launched from the Inbound Transaction Monitor, is the tool used to manage a specific inbound transaction. It is fully integrated into the inbound monitor. If approval workflow is activated for an inbound queue, all transactions checked into that queue must be reviewed and approved first using the Transaction Inspector tool.

Using the central tool, authorized users can perform the following tasks:

- view transaction data
- check-out/edit transaction data
- approve a transaction
- cancel a transaction

- process a transaction
- delete a transaction

7.5 Inbound Transaction Management Security

The back office users who manage and process the inbound transaction queues can access and modify client transaction data from various mobile users. Therefore, it is important that you define the appropriate authorization requirements for the back office users.

Inbound Queue Authorization

Authorization check is important for inbound queue process. Specific authorizations can be defined in the Configuration Panel for common inbound queue activities such as transaction check-out/edit, transaction approval, transaction cancellation, transaction processing, and transaction deletion.

Authorization check is handled by the inbound queue handler.

→ Tip

It is strongly recommended that you review and set the appropriate authorization checks for the inbound transaction queue, especially for [Checking Out](#), which allows the back office user to modify client transaction data before posting, and [Review & Approve](#), which allows the back office user to approve the transaction for posting when approval workflow is activated.

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.

About the icons:

- Links with the icon  : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
 - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
 - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon  : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

© 2024 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see <https://www.sap.com/about/legal/trademark.html> for additional trademark information and notices.